



GeDefense WP

OPEN CORE 8.1.0

Administration, Security and Architecture Manual

Setup | Operations | Modules | Recovery | Technical Blueprint

VisionGaiaTechnology

Documentation status: 2026-09-02

Basis: supplied source state GeDefense WP - Open Core 8.1.0

Contents

This manual is divided into five parts. Part IV contains the complete technical architecture map from `ARCHITECTURE.md`.

Part I - Installation and initial configuration

- System requirements
- Installation
- Setup Wizard - seven steps
- Recommended rollout in three stages

Part II - Administrator manual for protection modules

- AEGIS
- ZEUS
- CERBERUS
- PROMETHEUS
- TRINITY XDR
- MORPHEUS
- TITAN
- HADES
- AIRLOCK
- GHOST TRAP
- NEMESIS
- STYX
- CHRONOS
- Integrity Monitor & Scanner
- KEY VAULT
- THRONEGUARD
- LOGINPAGER
- ORACLE & Security Center
- GORGON
- Secure Downloads

Part III - Operations, maintenance and troubleshooting

- Testing after changes
- Common problems
- Backup & Recovery
- Privacy & Telemetry

Part IV - Complete technical blueprint

- System overview
- Global architecture tree
- File mapping
- Module documentation
- Dashboard
- CSS/UI
- API
- Data flows
- Shared/Core
- Architecture relationships
- File references
- Architecture observations

Part V - Sources and version note

Document status and reading guide

This manual describes **GeDefense WP - Open Core 8.1.0** based on the supplied plugin source code and the accompanying `ARCHITECTURE.md`. It combines three perspectives in a single document:

- 1. Setup and safe initial operation** - what to do after installation and in which order.
- 2. Administrator manual** - what the important dashboard areas and settings technically do, when they are useful, and where misconfiguration can impair the website.
- 3. Technical blueprint** - the complete architecture map with files, modules, dashboard mappings, APIs, data flows and dependencies.

Important: GeDefense intentionally operates fail-closed in several areas and uses strict blocking mechanisms. Features such as Hades, Titan, Zeus, ThroneGuard, Morpheus enforcement or Styx can block legitimate access or integrations when enabled without a testing and approval phase. Before production changes, a working backup and an alternative administrative access path should be available.

Document conventions

Label	Meaning
Recommended for initial setup	conservative starting configuration for first deployment
Check compatibility	may affect themes, plugins, REST endpoints, external APIs or login flows
Lockout risk	may lock out the administrator
Advanced	for administrators who understand and can test the technical impact
Code source	statement is derived from the current plugin implementation or dashboard UI

Part I - Installation and initial configuration

1. System requirements

The plugin header of the supplied version identifies **GeDefense WP - Open Core 8.1.0**. The project README specifies WordPress 6.0+ and PHP 8.1-8.4. The core architecture does not rely on external PHP packages; central functionality is loaded from the plugin itself.

In practice, the following should be in place before rollout:

- WordPress administrator access with `manage_options`.
- WordPress write access to the intended vault/upload area.
- Working HTTPS, especially when using HSTS, secure cookies, a sandbox origin or admission tokens.
- Backup of files and database.
- With a reverse proxy/CDN: correct client-IP resolution so that whitelists and bans use the actual administrator IP.
- For optional AI features: a dedicated Groq API key stored through the Key Vault.

2. Installation

1. Deploy the plugin directory to `wp-content/plugins/` or install it as a ZIP through WordPress.
2. Activate **GeDefense WP - Open Core**.
3. On activation, `class-vis-schema.php` enforces the required database schema. On administrator requests, GeDefense checks version states and may re-run schema migrations.
4. The **VGT Suite** appears in the WordPress backend. Until the setup assistant has been completed, GeDefense displays a notice prompting you to start the Setup Wizard.
5. Complete the Setup Wizard before manually enabling aggressive protection profiles or login obfuscation.

What happens during boot

The `gedefense-wp.php` entry point loads the bootstrapper and security core, binds the schema/vault components, and starts two protection phases. **Phase 1** is activated immediately and covers the early perimeter; **Phase 2** is loaded in the normal plugin lifecycle and integrates the dashboard and advanced protection modules, among other components. The technical sequence is documented in full in Part IV.

3. Setup Wizard - the seven steps

The current Setup Wizard is not merely a visual introduction; it writes actual module options. It also detects the administrator IP through the same proxy-aware resolver used by the protection modules and initially inserts it into the AEGIS and Prometheus fields when those fields are empty.

Step 1 - System check and IP whitelisting

Goal: prevent the current administrator session from being treated as an attacker by AEGIS or Prometheus.

Field	Effect
AEGIS & CERBERUS IP Whitelist	trusted IPs; the Wizard describes them as exempt from the firewall/ban system
Prometheus Scanner Whitelist	IPs whose activity should not be evaluated heuristically in the same way

Recommendation: Verify the detected IP. When using a VPN, Cloudflare, a reverse proxy or a dynamic address, do not accept it blindly; make sure GeDefense actually sees the intended client IP.

Lockout risk: The dashboard itself warns that missing AEGIS and Prometheus exemptions can block REST, nonce and update requests.

Step 2 - Firewall and WAF

By default, the Wizard enables AEGIS, Zeus and Cerberus if no explicit configuration exists yet.

AEGIS starts in **learning** mode by default. Learning logs and observes; **strict** is the aggressive Zero-Trust response with request termination and automatic ban coupling.

Zeus is preselected in the Wizard with the base firewall and 6G blacklist. Zeus operates earlier than the more complex WordPress layers and validates structural request limits, admission, host lock, budgets and route contracts.

Cerberus manages the fast IP-ban layer and is the target of several escalation paths from other modules.

Recommended for initial setup: AEGIS Learning, Zeus Balanced/Default, Cerberus enabled. Only after reviewing logs and whitelists should you switch AEGIS to Strict or use stricter Zeus profiles.

Step 3 - Malware protection and RASP

Prometheus is enabled initially in the Wizard. It evaluates behavior, frequency and threat scores.

Morpheus is also enabled initially, but its **Enforcement Mode** is not. This distinction is important: Audit/Learning allows you to observe call stacks and legitimate operations first. Strict enforcement should only be enabled after compatibility testing.

Step 4 - Deception and traps

Nemesis and **Ghost Trap** are enabled initially in the Wizard. Nemesis uses deception and canary mechanisms; Ghost Trap provides honeypot/decoy routes or lures whose access can trigger a defensive response.

Check compatibility: Your own security scanners, monitoring services or external penetration tests may intentionally touch such decoys. Authorized scanners should be accounted for in advance.

Step 5 - Hardening and stealth

Titan is enabled initially. By default, the Wizard enables XML-RPC protection and version obfuscation; REST blocking and feed disabling are not enabled automatically.

Hades is not enabled by default in the Wizard. If enabled, the assistant defines a secret parameter and a secret value. After saving, it displays the resulting secret administrator URL.

ThroneGuard and **LoginPager** are also not enabled automatically. This is intentional: ThroneGuard changes privileges, while LoginPager changes the login surface.

Step 6 - Autopilot, egress and AI

Chronos and **Styx** are enabled initially in the Wizard. Chronos runs scheduled integrity/filesystem checks. Styx controls outbound HTTP communication through a whitelist.

Optionally, a **Groq API key** can be stored. The Wizard describes it as encrypted in the Vault. AI features are additional diagnostics and do not replace the deterministic protection layers.

Step 7 - Arming the system

The final step summarizes the selected protection modules and stores the configuration. If Hades or a Zeus login path is active, the success screen displays the new administrator access paths.

Immediately afterwards:

1. Record the new administrator access path in a password manager.
2. Test the website in a private browser window.
3. Verify frontend, login, media upload, REST/AJAX functions and any planned external APIs.
4. Check Security Center and logs for unexpected blocking.
5. Only then tighten Strict/Enforcement profiles.

4. Recommended rollout in three stages

Stage A - Baseline

- AEGIS: **Learning**
- Zeus: **Balanced**, no incident-lockdown experiments
- Cerberus: enabled
- Prometheus: enabled, own administrator IP whitelisted
- Titan: enabled, initially with a compatible policy
- Morpheus: Audit/Learning, Enforcement off
- Chronos: enabled
- Key Vault: manage secrets only through the Vault

Stage B - Observation and compatibility

At minimum, test the typical workflows: login, page editing, media upload, plugin/theme updates, WooCommerce/REST where applicable, webhooks, external API calls, cron and backup software. Blocking should not be resolved by indiscriminately disabling a module; use the smallest necessary exception instead.

Stage C - Hardening

After a stable baseline, AEGIS Strict, stricter Zeus profiles, Titan enforcement, Morpheus enforcement, Hades and ThroneGuard can be enabled step by step. Change only one security-relevant variable at a time and re-test core functions afterwards.

Part II - Administrator manual for protection modules

5. AEGIS - Deep Packet Inspection WAF

Purpose

AEGIS inspects incoming data from GET, POST, JSON, headers, cookies and other request areas. The architecture describes recursive normalization and attack classes such as SQL injection, XSS, RCE, LFI and unsafe deserialization. The dashboard displays WAF state, Protection Mode, whitelists and Oracle diagnostics.

Important settings

Setting	Meaning	Recommendation
Engine enabled	turns AEGIS on/off	enabled
Learning	logs/observes instead of immediately escalating at maximum severity	initial operation
Strict	aggressive response, request termination/ban coupling	after baseline
Trusted IPs	IPs treated as administrator/trusted	verify own admin IP
Trusted User-Agents	exceptions by UA keyword	use sparingly
Oracle Neural Link	optional AI diagnostics	only with Vault key

Operational notes

Whitelists are security-critical: too broad means bypass; too narrow can block legitimate administrator actions. With a proxy/CDN, always verify client-IP resolution first. According to the UI, the Groq ping in the dashboard checks TLS/auth/API latency without transmitting prompts or content.

6. ZEUS - Pre-Boot Admission Kernel

Purpose

Zeus controls requests as early as possible. In the current version this includes host and canonical routing, structural request limits, route contracts, request budgets, admission tokens, lockdown states and a tamper-evident blackbox.

Security Profile

Profile	Character
BALANCED	recommended starting balance
STRICT	stronger protection, greater compatibility risk
PARANOID	maximum containment, only after testing
CUSTOM	manually defined policy

Emergency Lockdown State

NORMAL, HARDENED, FORTRESS and INCIDENT LOCKDOWN progressively increase access requirements. Incident Lockdown should be treated as an emergency state, not as a permanent default configuration.

Host Lock

- **DISABLED:** no host policy.
- **AUDIT:** log violations.
- **REJECT:** reject non-canonical hosts with 421.
- **Canonical Hosts:** enter one permitted host per line.

Check compatibility: Staging domains, alternative domains, internal health checks and proxy hostnames must be considered.

Request Budget Engine

The UI defaults are 180 requests/60s per IP and 450 requests/60s per subnet. Available actions include throttle, temporary 503 reject or XDR signal. These limits are not universal truths; shops, APIs, crawling or highly AJAX-heavy sites may require different profiles.

Brute-force and login hardening

The UI provides a Custom Login URL Slug, Magic Gateway Cookie, 404 Sweep Lockout, Failed Login Lockout and enforced administrator session timeouts. Always test and document login-path changes in a second browser.

Request Envelope Limits

The UI defaults include, among other values, a 2048-byte query length, 100 query parameters, 50 headers, 16 KiB total header size and an 8 KiB cookie size. A default POST limit of 64 MiB is prefilled in the form. Route Contracts can additionally enforce more granular rules per path.

Blackbox and Hardening Lab

The Blackbox collects early Zeus telemetry in a hash chain and can forward it to TRINITY XDR. The Hardening Lab provides benchmarks and self-tests; performance values are always host- and workload-dependent.

7. CERBERUS - Perimeter and ban layer

Cerberus is the fast blocking layer. The architecture maps APCu/Object Cache lookups, the ban table and exports for webserver/firewall rules to it. AEGIS, Prometheus and Ghost Trap can trigger bans.

Dashboard actions: view banned IPs, manually ban, unban and export rules. Before unbanning, review the reason in the event/XDR context.

8. PROMETHEUS - Behavioral Profiler

Prometheus evaluates not only individual payloads but behavior over time. In the dashboard, score decay, 404/auth/burst/method/parameter/regex penalties and Event Horizon thresholds can be configured.

Guideline

- Thresholds too low: false positives for administrators, crawlers and APIs.
- Thresholds too high: behavior-based detection loses effectiveness.

- Make changes based on real telemetry.
- Properly whitelist administrator and known monitoring IP addresses.

9. TRINITY XDR - Correlation and response

TRINITY connects events from different sensors into incidents and responses. The dashboard is organized into Situation, Incidents, Attack Stories, Entities, Responses, Evidence, Policy, Events and Health.

Key policy areas

- enable/disable automatic response;
- containment policy;
- escalation for repeat offenders;
- retention period;
- TTLs for Actor Ban, subnet, Zeus Route, Admission, Morpheus Overlay and Styx Overlay.

Principle: temporary, traceable containment is safer in normal operation than unnecessarily permanent blocking. Evidence and response history should be reviewed before manual escalation.

10. MORPHEUS - Runtime Application Self-Protection

Morpheus monitors PHP call contexts and protects sensitive operations at runtime. The architecture names DB shields for wp_users/wp_usermeta, a network shield against cloud metadata/SSRF and a path jail.

Audit first, enforcement later

The Setup Wizard does not automatically enable Morpheus enforcement. Keep it that way until typical plugins and workflows have been observed. In the dashboard, Audit Matrix entries and AI-assisted call-stack analyses can be approved or rejected.

Check compatibility: Plugins that legitimately modify users, usermeta, sensitive options or unusual network targets may be flagged in strict mode.

11. TITAN - Application & Browser Confinement

Titan is one of the most comprehensive modules and combines WordPress hardening with HTTP/browser policies and an assurance lifecycle.

Policy Profiles

Profile	UI rating	Use
Compatible	Low Breakage Risk	entry point/heterogeneous plugin landscape
Balanced	Medium Breakage Risk	default after compatibility testing
Strict	High Breakage Risk	controlled environments
Paranoid	Very High Breakage Risk	special cases/lab
Custom	Admin Defined	targeted custom policy

CSP and Origins

Titan maintains separate origins for Script, Style, Image, Connect and Frame. The more restrictive the CSP and origin lists are, the more likely external fonts, payment providers, maps, analytics, video embeds or API clients are to break. Titan therefore supports Learning/Observed Origins and a Candidate/Report-Only/Enforce process.

Assurance Lifecycle

The UI guides the administrator through **Learning -> Candidate -> Validate Candidate -> Report-Only -> Enforce** and provides rollback. This is the recommended path: do not enforce an unknown CSP immediately.

HSTS

- **HSTS enabled:** sends Strict-Transport-Security only over SSL.
- **max-age:** code constrains the value to between 300 and 63,072,000 seconds.
- **includeSubDomains:** includes subdomains.
- **preload:** is appended only together with includeSubDomains.

Caution: HSTS, especially includeSubDomains and preload, has effects beyond WordPress. Enable it only if all affected hosts can permanently use HTTPS.

Permissions Policy

Permissions allowed for self controls which browser features may be used by the site's own origin. Features that are not needed should not be broadly allowed.

XML-RPC

The implementation supports several modes: `disabled`, `pingback_disabled`, `auth_only`, `honeypot` and `custom`. The legacy `titan_block_xmlrpc` switch is internally mapped to an appropriate policy.

- **disabled:** disable XML-RPC.
- **pingback_disabled:** remove pingback methods while retaining other XML-RPC use.
- **honeypot:** record access to `xmlrpc.php` as a deception event and terminate with 403.

REST API and user enumeration

`titan_anti_enum` removes public user endpoints and blocks author enumeration. `titan_block_rest` restricts unauthenticated REST access but includes a list of allowed namespaces. This feature can affect headless frontends, forms, WooCommerce, external integrations or custom REST APIs.

Application Passwords

The implementation supports `allow`, `disable` and `audit`. Audit permits authentication and reports it to XDR; Disable turns off WordPress Application Passwords.

Server spoofing and camouflage

Titan can influence server headers and spoof generator metadata for Laravel, Drupal or Joomla. This is reconnaissance deception, not a substitute for patching. With upstream webserver/proxies, the final header may be overwritten outside PHP.

Cleanup options

- remove WordPress version;
- remove discovery links;
- remove emoji infrastructure;
- reduce oEmbed discovery;
- remove asset version query strings;
- disable Heartbeat.

Only disable Heartbeat after testing editor/plugin functionality that depends on it.

Fetch Metadata and browser policies

Titan can incorporate Fetch Metadata signals and additional browser security headers into the policy. The UI explicitly distinguishes support/observation from a true global enforcement guarantee.

Active Content Sandbox

Registered HTML, SVG and XML content can be rendered through signed preview URLs with a script-free sandbox CSP. A dedicated sandbox origin is accepted only when it uses HTTPS, is separate from the application host and is not privileged by a shared WordPress cookie domain.

Server Rules

According to the UI, Nginx rules are written only to a verified private vault outside the Document Root; otherwise they remain an export. **GENERATED/EXPORT ONLY does not mean VERIFIED ACTIVE**. Activation on the webserver must be verified separately.

12. HADES - Admin Cloaking

Hades hides standard admin/login paths behind a secret gate and can return 404 mimicry.

Settings

- Enable engine.
- Admin parameter, e.g. `vgt_access`.
- Secret, shown as `omega` in the Wizard example.
- additional stealth/server rules depending on dashboard version.

Lockout risk: After activation, save the generated secret admin URL and immediately test it in a private browser. Parameter and secret are access credentials and do not belong in tickets, screenshots or public documentation.

13. AIRLOCK - Upload protection

Airlock hooks into the WordPress upload path, checks file types/magic bytes and can sanitize SVG/XML or hand suspicious files to the scanner/quarantine.

Setting	Effect
Engine enabled	turns upload protection on
Cryptographic Filename Entropy	replaces/obfuscates filenames
Max Upload Size	hard upload limit
Extension Whitelist	allows only defined extensions

Check compatibility: WordPress editors, backup plugins or business workflows may require file formats that are not present in a narrow whitelist.

14. GHOST TRAP - Honeypots

The UI allows up to 50 traps, a list of polymorphic extensions and multiple name-generator profiles (Mixed, System Fakes, Backup Fakes, Random Hashes). According to the UI, saving regenerates the existing trap network.

Operation: Account for or whitelist external authorized scanners. Every hit is initially a signal; use Cerberus/XDR context for incident assessment.

15. NEMESIS - Deception Grid

Nemesis creates bounded deception: tarpits/delays, fake responses and cryptographic canaries. The architecture explicitly states that no hack-back is intended. Delays should remain bounded so that PHP workers are not intentionally exhausted.

16. STYX - Zero-Trust Egress

Styx intercepts outbound WordPress HTTP requests (`pre_http_request`) and enforces a domain/host whitelist. This protects against unwanted telemetry and exfiltration, but can also block legitimate APIs, update servers, payment services or webhooks.

Rollout: first inventory the destinations that are actually required. After activation, specifically test updates, license servers, mail/SMS providers, CAPTCHA, payment providers, AI APIs and custom webhooks.

17. CHRONOS - Background integrity

Chronos orchestrates scheduled scans. The dashboard provides, among other settings, an interval as well as recipient, subject and body text for warning emails. An interval that is too short can create unnecessary I/O load on large installations; an interval that is too long increases time to detection.

18. Integrity Monitor and scanner

The Integrity area manages baseline/hash comparisons and displays new, modified, deleted and malware-rated files. A baseline should only be updated/approved when the change is legitimate and understood.

Incident rule: Do not immediately accept an unexpected modification as the new baseline. First verify the source, plugin/core update, timestamp, diff and malware finding.

19. KEY VAULT - Secrets

The Key Vault stores API keys/secrets under unique identifiers. The UI describes AES-256-GCM, AAD binding and sealed assets. After entry, plaintext secrets should not also be duplicated in `wp_options`, notes or source code.

Deletion should be treated as irreversible. Before deleting, check which modules consume the identifier.

20. THRONEGUARD - Privilege Separation

ThroneGuard separates the GeDefense Master from normal administrators. It can remove administrator capabilities and enforce a Superkey-based backend/REST gate with a time-limited session.

Critical settings

- **Restrict Administrator role:** removes selected capabilities from normal administrators.

- **Lock Master Backend & REST API:** requires Superkey verification.
- **Superkey:** at least 12 characters according to the UI; when changing it, the current key is intended for verification.
- **Restricted Capabilities:** explicit matrix of removed permissions.

Lockout risk: Before activation, at least one correctly provisioned Master account and a securely stored Superkey must exist. Changes to `manage_options`, `plugin/theme/user` permissions can fundamentally alter administration workflows.

21. LOGINPAGER - Local login surface

LoginPager modifies the WordPress login surface locally. Background color, accent color, background image, logo, title and subtitle are configurable. The module is primarily UI/branding hardening and should be considered separately from Hades/Zeus access logic.

22. ORACLE and Security Center

Oracle performs a multi-vector security/posture audit and displays checks, status and analysis results. **Security Center** aggregates assurance, integrity checks, Trust Architecture, permission profiles and Application Confinement health.

An audit finding is a diagnosis, not automatic proof of a successful attack. Findings should be correlated with logs, XDR events and file integrity.

23. GORGON - Mesh Sync

Gorgon synchronizes threat indicators between GeDefense nodes. The dashboard manages Nexus URL, API key, nodes, ping and sync. Every mesh connection expands the trust boundary; use only controlled HTTPS destinations and Vault-protected secrets.

24. Secure Downloads

The Download Manager creates private, hash-verified copies and temporary download paths. Administrators should treat public IDs/links like temporary permissions and should not publish them permanently.

Part III - Operations, maintenance and troubleshooting

25. Testing after changes

After every significant policy change, at minimum:

1. Open the frontend as an anonymous user.
2. Perform a fresh administrator login.
3. Save a page/post.
4. Test media upload.
5. Test REST/AJAX functions of an important plugin.
6. Test an external API call/webhook if Styx is enabled.
7. Check Security Center, XDR and logs for new blocking.
8. With Titan CSP/browser policy, inspect the browser console and violation view.

26. Common problems

Administrator cannot log in

Check in this order: Hades URL/secret, Zeus Login Admission/Custom Route, ThroneGuard Superkey, AEGIS/Cerberus ban and administrator IP whitelist. Do not disable multiple protection modules at once; isolate the cause layer by layer.

REST or AJAX fails

Check Titan REST rules, AEGIS Strict, Prometheus score, Zeus Route Contracts and administrator IP exemptions. The dashboard notice explicitly cites missing AEGIS/Prometheus exemptions as a possible cause of REST/nonce/update problems.

External API does not work

Check the Styx whitelist and Titan `connect - src/origin` policy. For AI, also check the Vault key and Groq diagnostics.

Upload is rejected

Check Airlock extension whitelist, max upload, magic-byte/scanner finding and quarantine.

Frontend loses styles/scripts

Check Titan CSP, Observed Origins, Candidate/Report-Only and Enforce state. Do not immediately add global wildcards; add only the required origins.

27. Backup and recovery principles

- Back up the database and plugin files before policy or version changes.
- Store Hades/Zeus/ThroneGuard access credentials separately from the server.
- Never overwrite the Integrity baseline without review after an incident.
- Version exported server rules and syntax-check them before restarting the webserver.
- During rollback, identify the outermost blocking layer first: Zeus/Cerberus before WordPress-level policies.

28. Privacy and telemetry

The architecture contains redaction and pseudonymized export components. Nevertheless, an operator should define which security logs are stored, for how long, and who can access them. XDR retention and export data belong in the organization's own privacy/logging concept.

Part IV - Complete technical blueprint

The following part adopts the supplied architecture map as a technical reference. It organizes the code not only by directories, but also by boot phases, modules, dashboard views, APIs, data flows, styles, databases and dependencies. Paths are relative to the root directory of the supplied plugin.

System overview

GeDefense WP - Open Core (VisionGaia\GeDefense) is an uncompromising, autonomous web security and integrity operating system for WordPress, implemented as a full standard WordPress plugin (wp-content/plugins/gedefense-wp-core/).

The system follows a strict **Multi-Phase Ignition Pipeline**:

1. Phase 1: Perimeter Lockdown (Pre-Flight Kernel)

- Starts synchronously when the main plugin file `gedefense-wp.php` is loaded during the WordPress plugin-loading cycle.
- Initializes the XDR Event Fabric, mounts the file download manager and initializes the I18n matrix.
- Cerberus (Priority 0)**: O(1) memory lookup for banned IPs through APCu/Object Cache. On match: immediate HTTP 403 termination in < 0.1 ms.
- AEGIS (Priority 1)**: Deep Packet Inspection WAF for normalization and mitigation of SQLi, XSS, RCE, LFI and deserialization attacks.
- Striking Queue (Priority 2-N)**: Zeus (6G Pre-Boot WAF), Prometheus (Behavioral Profiling), Nemesis (Asymmetric Deception), Morpheus (RASP Hypervisor) and Gorgon (Telemetry Mesh).

2. Phase 2: Invariant & Hardening Subsystems

- Hooks into the standard `plugins_loaded` hook (Priority 10), or executes Phase 2 immediately if `plugins_loaded` has already fired.
- Starts the Compatibility Manager, Titan (browser & surface confinement), Hades (admin cloaking), Airlock (upload file airlock), Ghost Trap (honeypots), Chronos (background daemon), Styx (outbound Zero-Trust firewall), ThroneGuard (admin privilege isolation), LoginPager, Key Vault, Integration Bus and the dynamic Add-On Registry.
- In administrator context (`is_admin()`), initializes the **Dashboard subsystem** with glassmorphic UI, AJAX controllers and export pipelines.
- Uses standard WordPress lifecycle hooks: `register_activation_hook()` for automatic table creation (`VIS_Schema::enforce()`) and `register_deactivation_hook()` to clean up cron schedules and flush rewrite rules.

Core design invariants:

- Zero External Dependencies**: no Composer packages and no CDNs (Google Fonts, unpkg, jsdelivr).
- Fail-Closed**: critical core subsystems terminate with HTTP 503 when tampering is detected instead of falling back to an unsecured bypass.
- VGT DIAMANT SUPREME doctrine**: strict separation of markup, stylesheets and JavaScript. Zero inline styles, zero inline scripts, zero inline event handlers and zero DOM string sinks (`innerHTML`).

1. GLOBAL ARCHITECTURE TREE

GEDEFENSE WP - OPEN CORE (STANDARD WORDPRESS PLUGIN)	
— Entrypoints & Bootstrapping	
— <code>gedefense-wp.php</code>	(Standard plugin entrypoint, lifecycle hooks, constants)
— <code>class-vis-bootstrapper.php</code>	(Master Kernel Orchestrator: Phase 1 & 2 Ignition)
— <code>class-vis-schema.php</code>	(DB schema engine: 10 tables, dbDelta, Vault enforcement)
— <code>class-vis-vault.php</code>	(Hardware/software KMS: Libsodium secretbox & OpenSSL AES-256-GCM)
— Core Infrastructure (includes/core/)	
— <code>class-namespace-compatibility.php</code>	(Central ABI & legacy symbol aliasing)
— <code>class-vis-security.php</code>	(Zero-Trust utilities: pinned_https, client_ip, jailed_path)
— <code>class-vis-module-integrity.php</code>	(Cryptographic SHA-256 Merkle-tree invariant checker)
— <code>class-vis-module-registry.php</code>	(Dynamic Open-Core Add-On hub for VLP, Builder & SEO)
— <code>class-vis-i18n.php</code>	(Zero-overhead multilingual matrix DE / EN)
— <code>class-vis-event-bus.php</code>	(Standardized security-event emitter & legacy logging ABI)
— <code>class-vis-integration-bus.php</code>	(Event routing between sensors and XDR)
— <code>class-vis-security-health.php</code>	(Host environment health & PHP/server invariant checker)
— <code>class-vis-security-center.php</code>	(Central self-test & diagnostic reporter cockpit)
— <code>class-vis-ai-gateway.php</code>	(Isolated Groq LLM API transport adapter with memory bounds)
— <code>class-vis-trinity-grid.php</code>	(Interlock aggregator for Cerberus, Aegis, Prometheus, Nemesis)
— Layer 0 - 7 Defense Modules (includes/modules/)	
— <code>cerberus/</code>	(L0 perimeter drop, O(1) memory lookup, OS rule exporter)

└─ zeus/	(Pre-Boot 6G WAF, heuristic sanitizer & policy compiler)
└─ aegis/	(Deep Packet Inspection WAF, recursive token normalizer)
└─ prometheus/	(Cognitive behavior profiler, /24 subnet scoring, webshell
heuristics)	
└─ nemesis/	(Asymmetric Deception Grid, tarpits, decoy routing, frontend
canaries)	
└─ morpheus/	(Runtime Application Self-Protection / RASP stack hypervisor)
└─ gorgon/	(Encrypted node synchronization, peer-to-peer telemetry mesh)
└─ hades/	(Admin cloaking, path concealment & authentic 404 mimicry)
└─ airlock/	(Ingress upload streaming guard, magic-byte & polyglot scanner)
└─ trap/	(Ghost Trap: dynamic decoy files & immediate bans)
└─ styx/	(Zero-Trust egress whitelist guard, DNS rebinding protection)
└─ titan/	(Surface policy compiler, CSP, COOP/COEP, WP hardening)
└─ throneguard/	(Master/Admin privilege separation & Superkey session gate)
└─ loginpager/	(Zero-Trust local login surface & branded gateway)
└─ chronos/	(Autonomous background scheduler & file-integrity daemon)
└─ downloads/	(Secure Download Manager with private SHA-256-verified copies)
└─ vault/	(Key vault with AES-256-GCM & AAD-salt binding)
└─ oracle/	(12-vector host & vulnerability audit engine)
└─ filesystem/	(CHMOD permission & filesystem-integrity auditor)
└─ kernel/	(Pre-Boot WAF diagnostics & low-level sentinel)
─ TRINITY XDR Subsystem (includes/xdr/)	
└─ class-xdr-event-fabric.php	(Central XDR event bus & hook ingestion)
└─ class-xdr-event.php	(Typed, immutable XDR security-event model)
└─ class-xdr-event-repository.php	(High-performance storage for XDR events with deduplication)
└─ class-xdr-incident-engine.php	(Causal incident clustering & attack-story generator)
└─ class-xdr-policy-engine.php	(Rule-based deterministic incident evaluation)
└─ class-xdr-response-engine.php	(Autonomous, reversible defensive actions & rollbacks)
└─ class-xdr-evidence-store.php	(Cryptographic Merkle evidence chain for incidents)
└─ class-xdr-redactor.php	(GDPR-compliant bounded masking of sensitive parameters)
└─ class-xdr-request-context.php	(Deterministic request & correlation UUID binding)
─ Continuous Scanner & Malware Engine (includes/scanner/)	
└─ class-vis-scanner-engine.php	(Paginated, path-isolated Merkle-tree scanner)
└─ class-vis-malware-engine.php	(Zero-dependency lexical scanner for Airlock & Chronos)
└─ detectors/	(Specialized detection modules for PHP, SVG, archives, polyglots)
└─ storage/class-vis-quarantine-store.php	(Isolated private quarantine vault for threats)
└─ value/	(Typed value objects: Context, Budget, Finding, Verdict)
─ Command Dashboard UI (includes/dashboard/)	
└─ class-vis-dashboard-core.php	(Menu registration: vgt-suite, vgt-throneguard, vgt-loginpager)
└─ class-vis-dashboard-view.php	(Master glassmorphic renderer & SVG topbar router)
└─ class-vis-dashboard-settings.php	(Whitelist validation & module-scoped configuration)
└─ class-vis-dashboard-assets.php	(Deterministic conditional enqueue engine for CSS/JS)
└─ class-vis-dashboard-ajax.php	(Central AJAX hub: scanner, Zeus, unban, explorer, Add-Ons)
└─ class-vis-sentinel-export.php	(Pseudonymized zero-dependency JSON telemetry exporter)
└─ views/	(27 modular dashboard pages + sidebar + subviews)
─ Dynamic Add-On Subsystems (Open-Core module hub)	
└─ VLP/	(Vision Legal Pro: Shadow-Net, GDPR Proxy, Lingua AI)
└─ VisionGaiaSEO/	(VisionGaia SEO Architect: schema, redirects, geo engine)
└─ builder/	(VisionGaia PageBuilder: Elementor Migrator, Copilot)
└─ Bridge/	(Universal bridge adapters, contracts & service providers)
└─ compatibility/	(Third-party WAF & builder bridges: AIOS, Elementor, Divi)
─ Assets (assets/)	
└─ css/	(vis-dashboard.css, vis-dashboard-modern.css, vis-titan.css, etc.)
└─ js/	(vis-dashboard.js, vis-scanner-client.js, vis-security-center.js,
etc.)	
─ Cryptographic Trust Anchors (integrity/)	
└─ module-manifest.json	(SHA-256 Merkle digest matrix for all 27 core components)
─ Test & Regression Suite (scripts/)	
└─ 28 specialized PHP test scripts (liveness, WAF, XDR, Titan, Morpheus, namespace, manifest)	

2. MAP FILES TO THE ARCHITECTURE

2.1 Bootstrapping & Kernel Entry

- `gedefense-wp.php`: Primary standard plugin loader; checks double-load guards, declares global constants (`VIS_VERSION`, `VIS_MANIFEST_DIGEST`, `VIS_PRODUCT_NAME`, `VIS_PATH`, `VIS_URL`, `VIS_VAULT_DIR`), registers lifecycle hooks (`register_activation_hook`, `register_deactivation_hook`), enforces Phase 1 Pre-Flight and hooks Phase 2 into `plugins_loaded`.
- `class-vis-bootstrapper.php`: Core class `VIS_Bootstrapper`; controls `engage_phase_1()` (Sovereign Temp Dir, XDR boot, I18n, Cerberus, Trinity Grid, Aegis, module queue) and `engage_phase_2()` (Titan, Hades, Airlock, Chronos, Styx, ThroneGuard, Dashboard).
- `class-vis-schema.php`: Core class `VIS_Schema`; declares table definitions (`vis_apex_bans`, `vis_omega_logs`, `vis_oracle_patterns`, `vis_rate_limits`, `vis_xdr_events`, `vis_xdr_incidents`, `vis_xdr_incident_events`, `vis_xdr_responses`, `vis_xdr_evidence`, `vis_secure_downloads`) and enforces Vault file permissions (0700/0600).

- `class-vis-vault.php`: Core class `VIS_Vault`; implements Libsodium `sodium_crypto_secretbox` with OpenSSL `aes-256-gcm` fallback and HKDF key derivation from salt material.

2.2 Core Shared Infrastructure (`includes/core/`)

- `includes/core/class-namespace-compatibility.php`: `NamespaceCompatibility`; manages migrations and autoloading between the canonical `VisionGaia\GeDefense` namespace and legacy global symbols (`VIS_*`).
- `includes/core/class-vis-security.php`: `VIS_Security`; foundational security primitives (`pinned_https_get` with DNS-rebinding protection, `client_ip` with Cloudflare validation, `jailed_path`, `timing_safe_equals`).
- `includes/core/class-vis-module-integrity.php`: `VIS_Module_Integrity`; validates live file hashes against `integrity/module-manifest.json` and calculates the Merkle root hash.
- `includes/core/class-vis-module-registry.php`: `VIS_Module_Registry`; dynamically manages Add-On modules (VLP, Builder, SEO) and checks activation status and paths.
- `includes/core/class-vis-i18n.php`: `VIS_I18n`; localization engine with an integrated DE/EN dictionary and no Gettext overhead.
- `includes/core/class-vis-event-bus.php`: `VIS_Event_Bus`; standardized event emitter for security events; persists to `wp_vis_omega_logs` and forwards to TRINITY XDR.
- `includes/core/class-vis-integration-bus.php`: `VIS_Integration_Bus`; cross-module message bus for synchronous notification of sensors about attacks.
- `includes/core/class-vis-security-health.php`: `VIS_Security_Health`; invariant checks of the host system (OPcache, OpenSSL, Libsodium, CHMOD, memory limits).
- `includes/core/class-vis-security-center.php`: `VIS_Security_Center`; diagnostic reporter and aggregator for system status, integrity checks and logs.
- `includes/core/class-vis-ai-gateway.php`: `VIS_AI_Gateway`; abstraction layer for Groq LLM requests with strict memory bounds.
- `includes/core/class-vis-trinity-grid.php`: `VIS_Trinity_Grid`; priming and interlock mechanism for the four pillars (Cerberus, Aegis, Prometheus, Nemesis).

2.3 Layer 0 - 7 Defense Modules (`includes/modules/`)

- **Cerberus:**
 - `includes/modules/cerberus/class-vis-cerberus.php`: `VIS_Cerberus`; O(1) memory drop, APCu IP cache, export of Nginx/Apache/nftables deny lists.
- **Zeus:**
 - `includes/modules/zeus/class-vis-zeus.php`: `VIS_Zeus`; WAF core class.
 - `includes/modules/zeus/src/class-zeus-compiler.php`: Generates atomic RegEx DFAs.
 - `includes/modules/zeus/src/class-zeus-shield.php`: Pre-Boot filtering against 6G/author scans.
 - `includes/modules/zeus/src/class-zeus-blackbox.php`: Telemetry buffer for block decisions.
 - `includes/modules/zeus/src/class-zeus-admission.php`: Token-based WAF bypass admission.
 - `includes/modules/zeus/src/class-zeus-benchmark.php`: WAF performance and latency benchmarking.
 - `includes/modules/zeus/src/class-zeus-budget.php`: CPU and time-budget guard.
 - `includes/modules/zeus/src/class-zeus-contracts.php`: Interface definitions for WAF rules.
 - `includes/modules/zeus/src/class-zeus-edge.php`: Edge-rule compiler for reverse proxies.
 - `includes/modules/zeus/src/class-zeus-env.php`: Environment and host checks.
 - `includes/modules/zeus/src/class-zeus-envelope.php`: Request-envelope encapsulation.
 - `includes/modules/zeus/src/class-zeus-learning.php`: Heuristic learning of legitimate requests.
 - `includes/modules/zeus/src/class-zeus-policy-manager.php`: Management of active policies.
 - `includes/modules/zeus/src/class-zeus-vault-resolver.php`: Key resolution for WAF tokens.
 - `includes/modules/zeus/src/class-zeus-xdr-bridge.php`: Bridge for ingesting events into TRINITY XDR.
- **Aegis:**
 - `includes/modules/aegis/class-vis-aegis.php`: `VIS_Aegis`; Deep Packet Inspection engine (DPI). Recursive token normalization for GET, POST, JSON, multipart and headers.
 - `includes/modules/aegis/class-vis-aegis-oracle.php`: `VIS_Aegis_Oracle`; heuristic attack-pattern analyzer.
- **Prometheus:**
 - `includes/modules/prometheus/class-vis-prometheus.php`: `VIS_Prometheus`; behavior profiler, IP and subnet scoring, decay algorithm, PHP webshell scanner.
- **Nemesis:**
 - `includes/modules/nemesis/class-vis-nemesis.php`: `VIS_Nemesis`; asymmetric deception, tarpits, decoy routing, frontend HMAC canaries.
- **Morpheus:**
 - `includes/modules/morpheus/class-vis-morpheus.php`: `Vis_Morpheus`; RASP entry point.
 - `includes/modules/morpheus/src/class-morpheus-hypervisor.php`: Hypervisor for live stack traces.
 - `includes/modules/morpheus/src/class-morpheus-path-jail.php`: PHP path isolation.
 - `includes/modules/morpheus/src/class-morpheus-tracer.php`: Execution tracer for hook auditing.

- `includes/modules/morpheus/src/class-morpheus-ui.php`: Dashboard UI components.
- `includes/modules/morpheus/src/class-morpheus-dashboard.php`: AJAX handler for RASP approvals.
- `includes/modules/morpheus/src/class-morpheus-ai.php`: AI-assisted call-stack analysis.
- `includes/modules/morpheus/src/shields/class-morpheus-shield-db.php`: DML protection for `wp_users`.
- `includes/modules/morpheus/src/shields/class-morpheus-shield-network.php`: SSRF blocking for cloud metadata (169.254.169.254).
- `includes/modules/morpheus/src/shields/class-morpheus-shield-state.php`: State verification.
- **Gorgon:**
 - `includes/modules/gorgon/class-vis-gorgon.php`: `VIS_Gorgon`; P2P mesh-node orchestration.
 - `includes/modules/gorgon/src/class-gorgon-sync-engine.php`: Synchronization pipeline.
 - `includes/modules/gorgon/src/class-gorgon-harvester.php`: Collects local deny lists and threats.
 - `includes/modules/gorgon/src/class-gorgon-uplink.php`: Secured HTTPS uplink to peer nodes.
 - `includes/modules/gorgon/src/class-gorgon-config.php`: Configuration model for mesh networks.
 - `includes/modules/gorgon/src/class-vis-gorgon-ajax.php`: Dedicated AJAX controller for Gorgon.
- **Hades:**
 - `includes/modules/hades/class-vis-hades.php`: `VIS_Hades`; cloaks `/wp-admin` and `wp-login.php`, cryptographic HMAC cookie gates, 404 mimicry.
- **Airlock:**
 - `includes/modules/airlock/class-vis-airlock.php`: `VIS_Airlock`; hooks into `wp_handle_upload_prefilter`.
 - `includes/modules/airlock/src/class-airlock-scanner.php`: Magic-byte validation, polyglot detection.
 - `includes/modules/airlock/src/class-airlock-sanitizer.php`: XML/SVG sanitization against XSS/XXE.
 - `includes/modules/airlock/src/class-airlock-config.php`: File-type and size policies.
- **Ghost Trap:**
 - `includes/modules/trap/class-vis-ghost-trap.php`: `VIS_Ghost_Trap`; monitors decoy paths (`.env`, `.git`, `backup.sql`).
 - `includes/modules/trap/src/class-ghost-trap-engine.php`: Response generator and ban trigger.
 - `includes/modules/trap/src/class-ghost-trap-authenticator.php`: Token validation for legitimate scans.
 - `includes/modules/trap/src/class-ghost-trap-config.php`: Decoy-list configuration.
- **Styx:**
 - `includes/modules/styx/class-vis-styx.php`: `VIS_Styx`; hooks into `pre_http_request`. Zero-Trust egress whitelist with SSRF and DNS-rebinding protection.
- **Titan:**
 - `includes/modules/titan/class-vis-titan.php`: `VIS_Titan`; master hardening class.
 - `includes/modules/titan/src/class-titan-surface-resolver.php`: Detects context (Public, Login, Admin, REST, AJAX).
 - `includes/modules/titan/src/class-titan-policy-compiler.php`: Compiles CSP, Permissions-Policy, COOP, COEP and HSTS.
 - `includes/modules/titan/src/class-titan-policy-store.php`: Manages and versions security policies.
 - `includes/modules/titan/src/class-titan-runtime.php`: Header injection and conflict resolution.
 - `includes/modules/titan/src/class-titan-violation-collector.php`: Collects CSP reports through REST API.
 - `includes/modules/titan/src/class-titan-learning.php`: Heuristic learning of domain origins.
 - `includes/modules/titan/src/class-titan-server-rules.php`: Exports Nginx and Caddy header configurations.
 - `includes/modules/titan/src/class-titan-login-gate.php`: One-time token for login access.
 - `includes/modules/titan/src/class-titan-sandbox.php`: Isolated preview for active content.
 - `includes/modules/titan/src/class-titan-recovery.php`: WP-CLI emergency recovery for CSP.
 - `includes/modules/titan/src/class-titan-assurance.php`: Verifies security status of headers.
- **ThroneGuard:**
 - `includes/modules/throneguard/class-vis-throne-guard.php`: `VIS_Throne_Guard`; Master-administrator isolation, removal of toxic Super Admin privileges, fingerprint-bound Superkey sessions.
- **LoginPager:**
 - `includes/modules/loginpager/class-vis-loginpager.php`: `VIS_LoginPager`; local hardening and styling of the login page (`login_enqueue_scripts`, `login_headerurl`).
- **Chronos:**
 - `includes/modules/chronos/class-vis-chronos.php`: `VIS_Chronos`; WP-Cron autopilot.
 - `includes/modules/chronos/src/class-chronos-engine.php`: Runs periodic filesystem scans.
 - `includes/modules/chronos/src/class-chronos-scheduler.php`: Manages cron intervals (`vis_hourly_scan_event`).
 - `includes/modules/chronos/src/class-chronos-alerter.php`: Sends alert emails on integrity violations.
 - `includes/modules/chronos/src/class-chronos-config.php`: Thresholds and recipients.

- **Secure Downloads:**
 - `includes/modules/downloads/class-secure-download-manager.php`: `DownloadManager`; private, immutable file copies with SHA-256 integrity proof and temporary download routes.
- **Key Vault:**
 - `includes/modules/vault/class-vis-key-vault.php`: `VIS_Key_Vault`; stores sensitive API keys (e.g. AIOS, Groq, Cloudflare) encrypted with AES-256-GCM and AAD binding.
- **Oracle:**
 - `includes/modules/oracle/class-vis-oracle.php`: `VIS_Oracle`; 12-vector audit engine for detecting configuration errors and vulnerabilities.
- **Filesystem Guard:**
 - `includes/modules/filesystem/class-vis-filesystem-guard.php`: `VIS_Filesystem_Guard`; audits CHMOD permissions on `wp-config.php`, `.htaccess`, `/uploads/` and core directories.
- **Kernel Sentinel:**
 - `includes/modules/kernel/class-vis-kernel-sentinel.php`: `VIS_Kernel_Sentinel`; low-level monitoring of PHP limits and boot integrity.

2.4 TRINITY XDR Subsystem (`includes/xdr/`)

- `class-xdr-event-fabric.php`: Central ingestion channel for security events from all sensors.
- `class-xdr-event.php`: Immutable XDR security event with severity, category and actor hash.
- `class-xdr-event-repository.php`: CRUD and search operations on `wp_vis_xdr_events`.
- `class-xdr-incident-engine.php`: Correlates individual effects into attack incidents (`wp_vis_xdr_incidents`).
- `class-xdr-policy-engine.php`: Applies dynamic response policies to incidents.
- `class-xdr-response-engine.php`: Executes automated countermeasures (e.g. IP ban, session kill).
- `class-xdr-evidence-store.php`: Audit-safe Merkle evidence store in `wp_vis_xdr_evidence`.
- `class-xdr-redactor.php`: Removes passwords, credit-card data and sensitive information from log entries.
- `class-xdr-request-context.php`: Generates and binds request UUIDs and correlation IDs.

2.5 Scanner & Malware Engine (`includes/scanner/`)

- `class-vis-scanner-engine.php`: Paginated file scanner for the entire web root with baseline storage in `VIS_VAULT_DIR`.
- `class-vis-malware-engine.php`: Shared detector engine for syntactic and heuristic threat analysis.
- `detectors/class-vis-php-lexical-detector.php`: Token-based AST scanner for backdoors, obfuscation and `eval()`.
- `detectors/class-vis-polyglot-detector.php`: Checks images and archives for embedded PHP code.
- `detectors/class-vis-svg-xml-detector.php`: XML parser for detecting SVG-based XSS and XXE.
- `detectors/class-vis-archive-detector.php`: Inspects ZIP, TAR and GZ archives for suspicious content.
- `detectors/class-vis-path-context-detector.php`: Context-aware path and filename-pattern checks.
- `storage/class-vis-quarantine-store.php`: Isolates infected files in an encrypted directory.
- `contracts/interface-vis-file-detector.php`: Standardized interface for all file detectors.
- `value/class-vis-scan-budget.php`, `class-vis-scan-context.php`, `class-vis-scan-finding.php`, `class-vis-scan-verdict.php`: Typed data structures for scan runs.

2.6 Command Dashboard (`includes/dashboard/`)

- `class-vis-dashboard-core.php`: Menu registration and enqueue initialization.
- `class-vis-dashboard-view.php`: Layout rendering, SVG navigation and tab router.
- `class-vis-dashboard-settings.php`: Processes form submissions with nonce and type validation.
- `class-vis-dashboard-assets.php`: Manages conditional enqueue logic for 27 views and global assets.
- `class-vis-dashboard-ajax.php`: Central endpoint for AJAX actions (`vis_run_scan`, `vis_approve_changes`, etc.).
- `class-vis-sentinel-export.php`: Generates audit-safe JSON exports of system configuration and logs.
- `views/`:
 - `view-overview.php`: Control center & HUD.
 - `view-thread.php`: XDR threat matrix & event log.
 - `view-oracle.php`: Oracle system diagnostics.
 - `view-integrity.php`: Integrity checker & file explorer.
 - `view-security_center.php`: Consolidated Security Center (assurance, system status, logs).
 - `view-systatus.php`: Host & PHP infrastructure diagnostics.
 - `view-logs.php`: Searchable logbook.
 - `view-trinity.php`: Interlock matrix of the four protection shields.
 - `view-zeus.php`: Zeus WAF cockpit, compiler & benchmark.
 - `view-aegis.php`: AEGIS WAF configuration & ruleset.
 - `view-prometheus.php`: Prometheus heuristics & subnet monitor.
 - `view-cerberus.php`: IP ban list & ban management.

- `view-airlock.php`: Upload-filter configuration.
- `view-nemesis.php`: Tarpit & honeypot configuration.
- `view-ghost_trap.php`: Decoy-file configuration.
- `view-hades.php`: Admin cloaking paths & cloaking.
- `view-morpheus.php`: RASP call-stack inspector & shields.
- `view-titan.php`: Browser confinement & CSP compiler.
- `view-kernel.php`: Kernel status & low-level parameters.
- `view-styx.php`: Egress outbound whitelist.
- `view-chronos.php`: Background-scanner schedule & alerts.
- `view-vlp.php`: Privacy & Shadow-Net configuration.
- `view-filesystem.php`: Filesystem permissions.
- `view-vault.php`: Crypto key vault.
- `view-throneguard.php`: Master-admin privilege management.
- `view-loginpager.php`: Login-surface designer.
- `view-downloads.php`: Secure download files.
- `view-modules.php`: Add-On package manager.
- `view-setup_wizard.php`: Seven-step setup assistant.
- `view-xdr.php`: Detailed XDR incident situation view.
- `view-gorgon.php`: P2P mesh status & sync log.
- `view-sidebar.php`: Global sidebar component.
- `view-aos.php`: Legacy AIOS bridge status.
- `view-myrmidon.php`: Legacy Myrmidon device trust.

3. MODULE DOCUMENTATION

Module: Cerberus (L0 Perimeter Guard)

Purpose

Provides uncompromising Layer-0 perimeter protection that drops malicious requests in < 0.1 ms, before WordPress opens the database connection or loads themes/plugins.

Capabilities

- O(1) memory lookup of banned IPs through APCu and Object Cache.
- Immediate termination with HTTP 403 Forbidden and minimal CPU overhead.
- Dynamic banning when thresholds are exceeded by Aegis, Zeus, Prometheus or Ghost Trap.
- Automatic export of native webserver and firewall deny rules (`nginx_deny.conf`, `htaccess_deny.conf`, `nftables_drop.map`).

Associated files

```
Frontend / View:
- includes/dashboard/views/view-cerberus.php
- includes/dashboard/views/cerberus/script.js

Backend:
- includes/modules/cerberus/class-vis-cerberus.php

API:
- wp_ajax_vis_dashboard_unban_ip (in class-vis-dashboard-ajax.php)

Styles:
- includes/dashboard/views/cerberus/style.css

Database:
- wp_vis_apex_bans

Configuration:
- Option 'vis_config' (Keys: cerberus_enabled, cerberus_autoban)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Dashboard integration

- **Pages:** Dashboard -> `cerberus`

- **Components:** paginated ban table, unban modal, manual IP ban, webserver-rule download.
- **Styles:** cerberus/style.css
- **APIs:** wp_ajax_vis_dashboard_unban_ip
- **Backend functions:** VIS_Cerberus::instance()->unban_ip(), VIS_Cerberus::instance()->ban_ip()
- **Data:** list of banned IPs, reason, timestamp, request URI.
- **User actions:** manually ban an IP, unban an IP, download OS-firewall rules.

Dependencies

- Requires core: VIS_Security::client_ip()
- Reads/writes table: wp_vis_apex_bans
- Emits events to: VIS_Event_Bus, TRINITY XDR

Used by

- VIS_Bootstrapper::engage_phase_1() (Priority 0)
- VIS_Aegis (ban on severe exploit attempt)
- VIS_Prometheus (ban when the threat score exceeds the threshold)
- VIS_Ghost_Trap (immediate ban when a decoy file is accessed)
- VIS_Dashboard_Ajax (unban through the admin UI)

Module: Zeus (6G Pre-Boot WAF)

Purpose

Blocks malicious requests (malicious query strings, bad bots, author enumeration) using compiled deterministic regular expressions (DFA) before expensive PHP logic executes.

Capabilities

- High-speed inspection of request URI, user agent and referrer.
- Automatic compilation of 6G/7G signatures into optimized PHP RegEx trees (Zeus_Compiler).
- Admission-token generator for authorized scanners.
- Latency benchmarking and CPU budgeting.
- Seamless bridge for feeding block events into TRINITY XDR.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-zeus.php
- includes/dashboard/views/zeus/script.js

Backend:
- includes/modules/zeus/class-vis-zeus.php
- includes/modules/zeus/src/class-zeus-compiler.php
- includes/modules/zeus/src/class-zeus-shield.php
- includes/modules/zeus/src/class-zeus-blackbox.php
- includes/modules/zeus/src/class-zeus-admission.php
- includes/modules/zeus/src/class-zeus-benchmark.php
- includes/modules/zeus/src/class-zeus-budget.php
- includes/modules/zeus/src/class-zeus-config-repository.php
- includes/modules/zeus/src/class-zeus-contracts.php
- includes/modules/zeus/src/class-zeus-edge.php
- includes/modules/zeus/src/class-zeus-env.php
- includes/modules/zeus/src/class-zeus-envelope.php
- includes/modules/zeus/src/class-zeus-learning.php
- includes/modules/zeus/src/class-zeus-policy-manager.php
- includes/modules/zeus/src/class-zeus-vault-resolver.php
- includes/modules/zeus/src/class-zeus-xdr-bridge.php

API:
- wp_ajax_vis_save_zeus_config
- wp_ajax_vis_zeus_run_benchmark
- wp_ajax_vis_zeus_run_self_test
- wp_ajax_vis_zeus_drain_blackbox
- wp_ajax_vis_zeus_restore_preset
- wp_ajax_vis_zeus_save_contract
- wp_ajax_vis_zeus_delete_contract
- wp_ajax_vis_zeus_generate_admission_token
- wp_ajax_vis_zeus_rollback_policy

Styles:
- includes/dashboard/views/zeus/style.css

Database:
- Options: 'vis_zeus_config', 'vis_zeus_contracts'

Tests:
```

```
- scripts/zeus-env-regression.php
- scripts/zeus-live-waf-regression.php
- scripts/zeus-nextgen-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> zeus
- **Components:** telemetry tiles, Blackbox drain monitor, benchmark runner, Admission Token Manager, Policy Compiler Matrix.
- **Styles:** zeus/style.css
- **APIs:** 9 dedicated `vis_zeus_*` AJAX endpoints.
- **Backend functions:** `Zeus_Compiler::compile()`, `Zeus_Benchmark::run()`, `Zeus_Blackbox::drain()`
- **Data:** blocking statistics, latency in microseconds, active rulesets.
- **User actions:** tighten/relax WAF rules, run benchmark, generate emergency Admission Token, drain the Blackbox buffer.

Dependencies

- Requires core: `VIS_Security`, `VIS_Vault` (token signing)
- Feeds into: `TRINITY XDR` through `Zeus_Xdr_Bridge`
- Reads table: `wp_vis_rate_limits`

Used by

- `VIS_Bootstrapper::engage_phase_1()` (Priority 2)
- `VIS_Trinity_Grid` (state check)

Module: Aegis (Deep Packet Inspection WAF)

Purpose

Recursively inspects incoming payloads (GET, POST, JSON, XML, HTTP headers, cookies) up to 15 levels deep for SQL injection, Cross-Site Scripting (XSS), Remote Code Execution (RCE), Local File Inclusion (LFI) and unsafe deserialization.

Capabilities

- Two-stage pipeline: fast DFA signature matching followed by recursive normalization (resolving SQL comments, hex/Unicode homoglyphs, quote/slash slicing).
- Heuristic vector classification through `VIS_Aegis_Oracle`.
- Configurable whitelist for administrator IPs and known user agents.
- Seamless triggering of Cerberus bans on critical exploits.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-aegis.php
- assets/js/vis-oracle-diagnostics.js

Backend:
- includes/modules/aegis/class-vis-aegis.php
- includes/modules/aegis/class-vis-aegis-oracle.php

API:
- wp_ajax_vis_oracle_ping (diagnostic ping)

Styles:
- includes/dashboard/views/aegis/style.css

Database:
- Options: 'vis_config' (Keys: aegis_enabled, aegis_mode, aegis_whitelist_ips, aegis_whitelist_uas)
- Logs in: 'wp_vis_omega_logs'

Tests:
- scripts/aegis-regression.php
- scripts/sentinel-threat-benchmark.php
```

Dashboard integration

- **Pages:** Dashboard -> aegis
- **Components:** mode switch (Monitor / Enforce / Aggressive), whitelist editor for IPs and user agents, live inspection diagnostics.
- **Styles:** aegis/style.css
- **APIs:** `vis_oracle_ping`
- **Backend functions:** `VIS_Aegis::inspect()`, `VIS_Aegis_Oracle::analyze()`

- **Data:** blocking metrics, attack vectors, whitelist configuration.
- **User actions:** adjust protection mode, update whitelists, simulate test attacks against the WAF.

Dependencies

- Requires core: `VIS_Security::client_ip()`
- Triggers: `VIS_Cerberus::ban_ip()`, `VIS_Event_Bus::log()`
- Coupled with: `VIS_Trinity_Grid`

Used by

- `VIS_Bootstrapper::engage_phase_1()` (Priority 1)

Module: Prometheus (Behavioral Profiler & Heuristics)

Purpose

Monitors the behavior of IP addresses and /24 subnets in real time, accumulates threat scores across time windows and mitigates brute-force attacks and behavioral anomalies.

Capabilities

- Dynamic scoring based on request frequency, 404 generation and WAF hits.
- Mathematical score decay (harmless scores fade over time).
- Detection of malicious /24 subnet clustering attacks.
- Integrated PHP-webshell signature scanner for POST payloads.
- Atomic locking through MySQL `GET_LOCK` to prevent race conditions.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-prometheus.php
- includes/dashboard/views/prometheus/script.js

Backend:
- includes/modules/prometheus/class-vis-prometheus.php

API:
- Processes mutations through VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/prometheus/style.css

Database:
- Options: 'vis_config' (Keys: prometheus_enabled, prometheus_threshold, prometheus_whitelist_ips)
- Table: 'wp_vis_omega_logs' (Prometheus subsystem)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Dashboard integration

- **Pages:** Dashboard -> prometheus
- **Components:** anomaly gauge, subnet threat matrix, heuristic threshold control.
- **Styles:** prometheus/style.css
- **APIs:** form mutations through standard settings.
- **Backend functions:** `VIS_Prometheus::evaluate_request()`
- **Data:** current host risk assessment, blocked subnets, behavioral scores.
- **User actions:** configure sensitivity scales, maintain IP whitelists.

Dependencies

- Requires core: `VIS_Security::client_ip()`
- Calls: `VIS_Cerberus::ban_ip()` when the threshold is exceeded

Used by

- `VIS_Bootstrapper::engage_phase_1()` (Priority 3)

Module: Nemesis (Asymmetric Cyber Deception Grid)

Purpose

Creates asymmetric cost for attackers through bounded tarpits (micro-delays), forged API responses, manipulated status codes and injection of cryptographic frontend-tracking canaries.

Capabilities

- Bounded-delay tarpits without exhausting PHP worker threads.
- Deception responses (fake exploit success) for attackers while an alert is raised in the background.
- Injection of honeytokens (HMAC-SHA256 canary tokens) into HTML source.
- Strictly defensive approach: no hack-back payloads or response bombs (fully RFC- and law-compliant).

Associated files

```
Frontend / View:
- includes/dashboard/views/view-nemesis.php
- includes/dashboard/views/nemesis/script.js

Backend:
- includes/modules/nemesis/class-vis-nemesis.php

API:
- Settings through VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/nemesis/style.css

Database:
- Options: 'vis_config' (Keys: nemesis_enabled, nemesis_tarpit_delay, nemesis_fake_responses)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Dashboard integration

- **Pages:** Dashboard -> `nemesis`
- **Components:** Deception Grid overview, tarpit latency configurator, Canary Token Generator.
- **Styles:** `nemesis/style.css`
- **APIs:** form mutations through settings.
- **Backend functions:** `VIS_Nemesis::deploy_deception()`, `VIS_Nemesis::trap()`
- **Data:** number of deceived scanners, active honeytokens.
- **User actions:** adjust tarpit delay, enable fake routes.

Dependencies

- Requires core: `VIS_Security`, `VIS_Vault`
- Reports incidents to: `TRINITY XDR`, `VIS_Cerberus`

Used by

- `VIS_Bootstrapper::engage_phase_1()` (Priority 4)

Module: Morpheus (RASP Hypervisor & Execution Sandbox)

Purpose

Runtime Application Self-Protection (RASP): dynamically monitors PHP call stacks at runtime, isolates the database from unauthorized manipulation of the `wp_users` table and prevents Server-Side Request Forgery (SSRF) against cloud metadata.

Capabilities

- Live stack-trace inspection to validate calling plugins and scripts.
- Database Shield: intercepts DML operations (`UPDATE`, `DELETE`, `INSERT`) on `wp_users` and `wp_usermeta` and enforces cryptographic authorization tokens.
- Network Shield: blocks outbound requests to cloud metadata (`169.254.169.254`) and private IP ranges.
- Filesystem Jail: prevents unauthorized file operations outside the web root.
- AI-assisted call-stack anomaly detection through the Groq AI Gateway.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-morpheus.php
```

```
- includes/dashboard/views/morpheus/script.js

Backend:
- includes/modules/morpheus/class-vis-morpheus.php
- includes/modules/morpheus/src/class-morpheus-hypervisor.php
- includes/modules/morpheus/src/class-morpheus-path-jail.php
- includes/modules/morpheus/src/class-morpheus-tracer.php
- includes/modules/morpheus/src/class-morpheus-ui.php
- includes/modules/morpheus/src/class-morpheus-dashboard.php
- includes/modules/morpheus/src/class-morpheus-ai.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-db.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-network.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-state.php

API:
- wp_ajax_vgt_morpheus_trigger_ai
- wp_ajax_vgt_morpheus_reject_ai
- wp_ajax_vgt_morpheus_approve_ai
- wp_ajax_vgt_morpheus_delete_matrix
- wp_ajax_vgt_morpheus_toggle_strict

Styles:
- includes/dashboard/views/morpheus/style.css

Database:
- Options: 'vgt_morpheus_audit_matrix', 'vis_config'

Tests:
- scripts/morpheus-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> morpheus
- **Components:** RASP Hypervisor status, Hook Audit Matrix, DML block monitor, AI stack analysis.
- **Styles:** morpheus/style.css
- **APIs:** 5 dedicated AJAX endpoints for RASP approvals and AI triggers.
- **Backend functions:** `Vis_Morpheus::engage()`, `Morpheus_Shield_DB::intercept()`
- **Data:** audited PHP call stacks, detected injection attempts.
- **User actions:** authorize/block plugins in the Audit Matrix, switch Strict Mode.

Dependencies

- Requires core: `VIS_AI_Gateway`, `VIS_Security`
- Integrates with: `$wpdb` query filters

Used by

- `VIS_Bootstrapper::engage_phase_1()` (Priority 5)

Module: Gorgon (Encrypted Mesh Sync & Nexus Uplink)

Purpose

Enables decentralized, cryptographically secured node-to-node synchronization of IP deny lists and threat intelligence between multiple GeDefense instances.

Capabilities

- P2P mesh uplink with HMAC-SHA256 signature validation.
- Automatic export and import of indicators of compromise (IOCs).
- Isolated AJAX controller to avoid header collisions.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-gorgon.php
- includes/dashboard/views/gorgon/script.js

Backend:
- includes/modules/gorgon/class-vis-gorgon.php
- includes/modules/gorgon/src/class-gorgon-sync-engine.php
- includes/modules/gorgon/src/class-gorgon-harvester.php
- includes/modules/gorgon/src/class-gorgon-uplink.php
- includes/modules/gorgon/src/class-gorgon-config.php
- includes/modules/gorgon/src/class-vis-gorgon-ajax.php

API:
- wp_ajax_vgt_gorgon_toggle
- wp_ajax_vgt_gorgon_update_config
- wp_ajax_vgt_gorgon_ping_nexus
- wp_ajax_vgt_gorgon_sync
```

```

- wp_ajax_vgt_gorgon_add_node
- wp_ajax_vgt_gorgon_remove_node

Styles:
- includes/dashboard/views/gorgon/style.css

Database:
- Options: 'vis_config' (Keys: gorgon_enabled, gorgon_nexus_url, gorgon_api_key)

Tests:
- scripts/integration-regression.php

```

Dashboard integration

- **Pages:** Dashboard -> gorgon (reachable through a direct tab link)
- **Components:** Nexus node status, synchronization history, peer-node management.
- **Styles:** gorgon/style.css
- **APIs:** 6 dedicated vgt_gorgon_* AJAX routes.
- **Backend functions:** Gorgon_Sync_Engine::sync(), Gorgon_Uplink::ping()
- **Data:** connected nodes, latency, exchanged IP bans.
- **User actions:** add/remove nodes, manually trigger synchronization.

Dependencies

- Requires core: VIS_Security::pinned_https_get(), VIS_Vault

Used by

- VIS_Bootstrapper::engage_phase_1() (Priority 6)

Module: Hades (Admin Cloaking & 404 Mimicry)

Purpose

Cloaks administrative paths (/wp-admin, wp-login.php), blocks unauthorized direct access with convincing webserver-style 404 pages and authorizes administrators only through secret GET parameters or HMAC-secured cookies.

Capabilities

- Complete hiding of the standard login URL.
- Timing-safe HMAC-SHA256 gate-cookie validation.
- Automatic fallback to genuine Apache/Nginx 404 status behavior for unauthorized access.
- Compilation of native webserver rewrite rules for Nginx and Apache.

Associated files

```

Frontend / View:
- includes/dashboard/views/view-hades.php

Backend:
- includes/modules/hades/class-vis-hades.php

API:
- Mutations through VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/hades/style.css

Database:
- Options: 'vis_config' (Keys: hades_enabled, hades_secret_param, hades_secret_value, hades_stealth_mode)

Tests:
- scripts/hades-shadow-net-regression.php

```

Dashboard integration

- **Pages:** Dashboard -> hades
- **Components:** Secret URL Generator, bypass-link display, Nginx/Apache rewrite-rule box.
- **Styles:** hades/style.css
- **APIs:** configuration storage.
- **Backend functions:** VIS_Hades::protect_admin(), VIS_Hades::get_nginx_rules()
- **Data:** current secret key, authorized admin sessions.
- **User actions:** configure secret parameter, copy rewrite rules.

Dependencies

- Requires core: `VIS_Security::validate_hades_gate()`

Used by

- `VIS_Bootstrapper::engage_phase_2()`

Module: Airlock (Ingress Polyglot & Upload Sanitizer)

Purpose

Inspects file uploads in real time through `wp_handle_upload_prefilter`, detects polyglot files (for example PHP code in image headers) and neutralizes SVG XSS/XXE payloads.

Capabilities

- Validation of real MIME types using magic bytes rather than file extensions.
- Full XML parsing of SVG files to remove `<script>`, `onload` and external entities (XXE).
- Detection of embedded PHP tags (`<?php`, `<?=<script language="php">`) in image files.
- Automatic quarantine of suspicious uploads in the isolated vault.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-airlock.php

Backend:
- includes/modules/airlock/class-vis-airlock.php
- includes/modules/airlock/src/class-airlock-scanner.php
- includes/modules/airlock/src/class-airlock-sanitizer.php
- includes/modules/airlock/src/class-airlock-config.php

API:
- Filter: 'wp_handle_upload_prefilter'

Styles:
- includes/dashboard/views/airlock/style.css

Database:
- Options: 'vis_config' (Keys: airlock_enabled, airlock_quarantine_enabled, airlock_max_file_size)

Tests:
- scripts/malware-scanner-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> airlock
- **Components:** upload-policy table, SVG protection status, quarantine settings.
- **Styles:** `airlock/style.css`
- **APIs:** standard settings mutation.
- **Backend functions:** `VIS_Airlock_Scanner::scan_file()`, `VIS_Airlock_Sanitizer::sanitize_svg()`
- **Data:** blocked uploads, quarantine status.
- **User actions:** define permitted file types, enable quarantine.

Dependencies

- Uses scanner: `VIS_Malware_Engine`, `VIS_Quarantine_Store`

Used by

- `VIS_Bootstrapper::engage_phase_2()`
- `VIS_Titan` (upload-certificate validation)

Module: Ghost Trap (Dynamic Decoy Honeypots)

Purpose

Places dynamic decoy files and routes (`.env`, `.git/config`, `wp-config.php.bak`, `phpinfo.php`) on the website that legitimate users should never request, and immediately bans scanners that access them.

Capabilities

- Intercepts requests for highly sensitive filenames.

- Triggers an immediate, permanent Cerberus ban for the attacker's IP.
- Serves deceptive but harmless fake content.
- No dependency on real files on disk; routing is entirely virtual.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-ghost_trap.php

Backend:
- includes/modules/trap/class-vis-ghost-trap.php
- includes/modules/trap/src/class-ghost-trap-engine.php
- includes/modules/trap/src/class-ghost-trap-authenticator.php
- includes/modules/trap/src/class-ghost-trap-config.php

API:
- Mutations through VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/ghost_trap/style.css

Database:
- Options: 'vis_config' (Keys: ghost_trap_enabled, ghost_trap_exts)

Tests:
- scripts/ghost-trap-authentication-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> ghost_trap
- **Components:** decoy matrix, hit statistics, honeypot configurator.
- **Styles:** ghost_trap/style.css
- **Backend functions:** Ghost_Trap_Engine::evaluate()
- **Data:** number of triggered traps, banned IPs.
- **User actions:** add additional decoy files.

Dependencies

- Triggers: VIS_Cerberus::ban_ip(), TRINITY XDR

Used by

- VIS_Bootstrapper::engage_phase_2()

Module: Styx (Zero-Trust Egress Whitelist Guard)

Purpose

Monitors all HTTP and HTTPS connections originating from the server (`pre_http_request`) and prevents data exfiltration, Command-and-Control (C2) callbacks and DNS rebinding.

Capabilities

- Strict whitelist of permitted destination domains (WordPress.org updates, GeDefense Nexus, licensed APIs).
- Blocking of all connections to private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.1, cloud metadata).
- DNS-rebinding verification before connection establishment.
- Configurable mode: Audit-Only (logging) or Enforce (active blocking).

Associated files

```
Frontend / View:
- includes/dashboard/views/view-styx.php
- includes/dashboard/views/styx/script.js

Backend:
- includes/modules/styx/class-vis-styx.php

API:
- Filter: 'pre_http_request'

Styles:
- includes/dashboard/views/styx/style.css

Database:
- Options: 'vis_config' (Keys: styx_enabled, styx_mode, styx_whitelist)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Dashboard integration

- **Pages:** Dashboard -> `styx`
- **Components:** egress traffic monitor, whitelist manager, blocking log.
- **Styles:** `styx/style.css`
- **APIs:** standard settings mutation.
- **Backend functions:** `VIS_Styx::intercept_request()`, `VIS_Styx::is_whitelisted()`
- **Data:** outbound HTTP requests, blocked exfiltration attempts.
- **User actions:** approve external destination domains, switch mode.

Dependencies

- Requires core: `VIS_Security::validate_public_http_url()`
- Reports incidents to: `VIS_Event_Bus`, `TRINITY XDR`

Used by

- `VIS_Bootstrapper::engage_phase_2()`

Module: Titan (Surface Policy Compiler & Browser Confinement)

Purpose

Comprehensive hardening and surface-protection system. Compiles dynamic, context-aware browser security policies (CSP, Fetch Metadata, Permissions Policy, COOP, COEP, HSTS) and protects sensitive WordPress endpoints (XML-RPC, REST author enumeration, user discovery).

Capabilities

- Context resolution for 9 surfaces (`VIS_Titan_Surface_Resolver`): Public, Login, Admin, GeDefense Cockpit, REST, AJAX, Cron, Webhook, Active Preview.
- Deterministic compiler for Content Security Policy (CSP Level 3), Strict-Transport-Security (HSTS) and cross-origin policies.
- Strict 5-stage lifecycle model: Candidate Validation -> Report-Only -> Eligibility Check -> Enforce Confirmation -> Rollback.
- Automatic ingestion of CSP violation reports through the internal REST API `/wp-json/visiongaia/v1/titan/csp-report`.
- Native server-rule generation for Nginx and Caddy.
- Integrated WP-CLI emergency recovery (`VIS_Titan_Recovery`).
- Disabling of XML-RPC, author enumeration, XML-RPC honeypot, emoji scripts and version fingerprints.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-titan.php
- assets/js/vis-titan-command-center.js

Backend:
- includes/modules/titan/class-vis-titan.php
- includes/modules/titan/src/class-titan-surface-resolver.php
- includes/modules/titan/src/class-titan-policy-compiler.php
- includes/modules/titan/src/class-titan-policy-store.php
- includes/modules/titan/src/class-titan-runtime.php
- includes/modules/titan/src/class-titan-assurance.php
- includes/modules/titan/src/class-titan-learning.php
- includes/modules/titan/src/class-titan-violation-collector.php
- includes/modules/titan/src/class-titan-server-rules.php
- includes/modules/titan/src/class-titan-login-gate.php
- includes/modules/titan/src/class-titan-sandbox.php
- includes/modules/titan/src/class-titan-recovery.php

API:
- admin_post_vis_titan_policy_action
- admin_post_vis_titan_download_nginx
- admin_post_vis_titan_generate_gate_link
- admin_post_vis_titan_preview_link
- REST: POST /wp-json/visiongaia/v1/titan/csp-report

Styles:
- assets/css/vis-titan.css

Database:
- Options: 'vis_config' (Titan Keys), 'vis_titan_policies', 'vis_titan_violations'

Tests:
- scripts/titan-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> titan
- **Components:** policy-lifecycle control, CSP directive editor, violation viewer, server-config download.
- **Styles:** assets/css/vis-titan.css
- **APIs:** admin_post_vis_titan_policy_action, REST report ingress.
- **Backend functions:** VIS_Titan_Runtime::boot(), VIS_Titan_Policy_Compiler::compile()
- **Data:** active browser headers, Report-Only hits, blocked domain origins.
- **User actions:** activate/test/roll back policies, download server rules.

Dependencies

- Requires core: VIS_Security
- Integrates with: AirLock (certificate validation)

Used by

- VIS_Bootstrapper::engage_phase_2()

Module: ThroneGuard (Master-Role Privilege Separation)

Purpose

Strictly separates the Super Administrator role (Master) from conventional WordPress administrators, removes risky capabilities from normal administrators (plugin installation, file editing) and secures privileged actions through cryptographic Superkey sessions.

Capabilities

- Establishes a Master role with exclusive control over security modules.
- Restricts toxic capabilities for normal administrators (install_plugins, edit_plugins, edit_files, update_core).
- Session Superkey mechanism with time limits and hardware/fingerprint binding.
- Immutable audit logging of all administrative privilege activity.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-throneguard.php
- includes/dashboard/views/throneguard/script.js

Backend:
- includes/modules/throneguard/class-vis-throne-guard.php

API:
- admin_post_vis_throneguard_claim
- admin_post_vis_throneguard_save
- admin_post_vis_throneguard_unlock
- wp_ajax_vis_throneguard_clear_logs

Styles:
- includes/dashboard/views/throneguard/style.css

Database:
- Options: 'vis_throneguard_config', 'vis_throneguard_audit_log'

Tests:
- scripts/throneguard-loginpager-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> throneguard (also standalone submenu vgt-throneguard)
- **Components:** Master Claim Wizard, role matrix, Superkey unlock modal, privilege audit log.
- **Styles:** throneguard/style.css
- **APIs:** 3 admin-post actions + 1 AJAX clear action.
- **Backend functions:** VIS_Throne_Guard::handle_claim_master(), VIS_Throne_Guard::handle_unlock()
- **Data:** Master user ID, Superkey validity, role permissions.
- **User actions:** claim Master role, restrict administrator capabilities, unlock Superkey session.

Dependencies

- Requires core: VIS_Vault, VIS_Security

Used by

- VIS_Bootstrapper::engage_phase_2()

- `VIS_Dashboard_Core::register_menu_matrix()`

Module: LoginPager (Zero-Trust Login Surface)

Purpose

Provides a fully local, hardened and visually modernized WordPress login surface with responsive live preview in the admin area and zero external CDN dependencies.

Capabilities

- Modern glassmorphic design for the standard login page (`wp-login.php`).
- Protocol-validated image and logo embedding (`safe_url()` with HTTP/HTTPS whitelist).
- Preset color schemes: Cyber Cyan, Emerald Matrix, Purple Haze, Apex Gold, Crimson Core.
- Complete independence from external fonts or JavaScript frameworks.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-loginpager.php
- assets/js/vis-loginpager-admin.js

Backend:
- includes/modules/loginpager/class-vis-loginpager.php

API:
- Mutations through VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/loginpager/style.css

Database:
- Options: 'vis_config' (Keys: loginpager_enabled, loginpager_title, loginpager_subtitle, loginpager_bg_color, loginpager_accent, loginpager_bg_image, loginpager_logo)

Tests:
- scripts/throneguard-loginpager-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> loginpager (also standalone submenu `vgt-loginpager`)
- **Components:** dual-column customizer with live preview canvas, color picker with hex synchronization, preset color swatches.
- **Styles:** `loginpager/style.css`
- **APIs:** form mutations through standard settings.
- **Backend functions:** `VIS_LoginPager::render_login_page()`
- **Data:** design tokens, logo URLs, branding text.
- **User actions:** customize colors, background images and titles and inspect them live in the preview.

Dependencies

- Requires core: `VIS_Security`

Used by

- `VIS_Bootstrapper::engage_phase_2()`
- `VIS_Dashboard_Core::register_menu_matrix()`

Module: Chronos (Autonomous Background Integrity Daemon)

Purpose

Runs autonomous background checks of the entire filesystem and system configuration through WP-Cron (`vis_hourly_scan_event`) and sends alert notifications when tampering is detected.

Capabilities

- Automatic start of periodic integrity and malware scans.
- Resource-conscious execution with CPU budgeting.
- Sends alert emails when suspicious modifications are detected.
- Self-healing of interrupted scan states.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-chronos.php

Backend:
- includes/modules/chronos/class-vis-chronos.php
- includes/modules/chronos/src/class-chronos-engine.php
- includes/modules/chronos/src/class-chronos-scheduler.php
- includes/modules/chronos/src/class-chronos-alerter.php
- includes/modules/chronos/src/class-chronos-config.php

API:
- Hook: 'vis_hourly_scan_event'

Styles:
- includes/dashboard/views/chronos/style.css

Database:
- Options: 'vis_config' (Keys: chronos_enabled, chronos_frequency, chronos_email_recipient, chronos_email_body)

Tests:
- scripts/scanner-resumption-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> chronos
- **Components:** scheduler status, cron-interval configurator, email alert-template editor.
- **Styles:** chronos/style.css
- **Backend functions:** Chronos_Engine::run(), Chronos_Alerter::send_alert()
- **Data:** timestamp of the last scan, scheduled next execution, alert recipient.
- **User actions:** set scan frequency, edit email recipient and template.

Dependencies

- Uses subsystem: VIS_Scanner_Engine_Omega

Used by

- VIS_Bootstrapper::engage_phase_2()

Module: Secure Downloads (Attested Storage & Verifiable Links)

Purpose

Enables secure distribution of media files and software builds through temporary, cryptographically attested GeDefense download links with strict rate limiting and protection against unauthorized direct linking.

Capabilities

- Creates an immutable, isolated copy of the file in protected storage (VIS_VAULT_DIR).
- Calculates and verifies the SHA-256 hash on every download request.
- Serves files through `template_redirect` (-100) using the URL `/?vis_download={public_id}` without executing PHP files.
- Rate limiting per download ID to mitigate DoS.
- Targeted AEGIS WAF exemption only for the validated download path.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-downloads.php

Backend:
- includes/modules/downloads/class-secure-download-manager.php

API:
- Public URL: /?vis_download={public_id}
- admin_post_vis_download_register
- admin_post_vis_download_toggle
- admin_post_vis_download_delete

Styles:
- includes/dashboard/views/downloads/style.css

Database:
- Table: 'wp_vis_secure_downloads'

Tests:
- scripts/download-manager-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> downloads
- **Components:** registration form for Media Library files, table of all active downloads with click-to-copy function.
- **Styles:** downloads/style.css
- **APIs:** 3 admin-post actions for registration, enable/pause and deletion.
- **Backend functions:** DownloadManager::register(), DownloadManager::serve()
- **Data:** filename, integrity hash, download counter, GeDefense link.
- **User actions:** protect file, copy download link, pause/delete download.

Dependencies

- Requires core: VIS_Schema, VIS_Security
- Writes to table: wp_vis_secure_downloads

Used by

- VIS_Bootstrapper::engage_phase_1()
- VIS_Aegis (specific trusted path)

Module: Key Vault (Authenticated AES-256-GCM KMS)

Purpose

Acts as a secure, hardware- and server-bound key store (Key Management Service) for external API tokens and secrets using AES-256-GCM encryption and AAD salt binding.

Capabilities

- Authenticated Encryption with Associated Data (AAD) against ciphertext substitution.
- Key derivation using HKDF-SHA256 from persistent Vault material.
- Strict isolation of sensitive keys in the filesystem with 0600 permissions.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-vault.php

Backend:
- includes/modules/vault/class-vis-key-vault.php

API:
- Post actions in the view: vis_vault_save, vis_vault_delete

Styles:
- includes/dashboard/views/vault/style.css

Database:
- Options: 'vis_key_vault_registry'

Tests:
- scripts/security-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> vault
- **Components:** secret entry form, list of sealed key assets with delete function.
- **Styles:** vault/style.css
- **Backend functions:** VIS_Key_Vault::seal(), VIS_Key_Vault::unseal()
- **Data:** key identifier, encryption mode (AES-256-GCM), status (SEALED).
- **User actions:** seal a new secret, irreversibly delete an existing secret.

Dependencies

- Requires core: VIS_Vault

Used by

- VIS_Bootstrapper::engage_phase_2()
- VisionGaiaSE0\VaultBridge

Module: Oracle (12-Vector Security Audit Engine)

Purpose

Performs in-depth static and dynamic security analysis of the WordPress installation and detects misconfiguration across 12 key areas.

Capabilities

- Checks file permissions, SSL configuration, debug logs, user privileges, database prefixes and outdated software.
- Generates structured audit findings with remediation recommendations.
- Integrated interface for AI-assisted vulnerability assessments.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-oracle.php

Backend:
- includes/modules/oracle/class-vis-oracle.php

API:
- wp_ajax_vis_oracle_ping

Styles:
- includes/dashboard/views/oracle/style.css

Database:
- Table: 'wp_vis_oracle_patterns'

Tests:
- scripts/dashboard-consolidation-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> oracle
- **Components:** 12-vector audit matrix, vector status cards (Pass/Fail), remediation recommendations.
- **Styles:** oracle/style.css
- **APIs:** wp_ajax_vis_oracle_ping
- **Backend functions:** VIS_Oracle::get_audit_findings()
- **Data:** system integrity score, vector findings.
- **User actions:** manually trigger scan, analyze vulnerabilities.

Dependencies

- Requires core: VIS_Security_Health

Used by

- VIS_Bootstrapper::engage_phase_1()

Module: Filesystem Guard (Permission & CHMOD Invariant Auditor)

Purpose

Monitors filesystem permissions of critical WordPress components and raises an alert when writable permissions are assigned to sensitive files.

Capabilities

- Checks CHMOD permissions of wp-config.php (expected 0400/0440/0600), .htaccess and directories.
- Detects unprotected directories without index.php or .htaccess.

Associated files

```
Frontend / View:
- includes/dashboard/views/view-filesystem.php

Backend:
- includes/modules/filesystem/class-vis-filesystem-guard.php

Styles:
- includes/dashboard/views/filesystem/style.css
```

Dashboard integration

- **Pages:** Dashboard -> filesystem
- **Components:** file-permission table with status badges (Secure / Warning).

- **Styles:** filesystem/style.css
- **Backend functions:** VIS_FileSystem_Guard::audit()
- **Data:** path, current CHMOD, recommended CHMOD, security status.

Dependencies

- Requires core: VIS_Security::jailed_path()

Used by

- VIS_Bootstrapper::engage_phase_2()

Module: Kernel Sentinel (Runtime Invariant Monitor)

Purpose

Monitors the low-level PHP runtime environment and kernel invariants (memory limit, max execution time, OPcache status) and provides early warnings about resource constraints.

Capabilities

- Monitoring of memory usage and peak-memory values.
- Checks PHP extensions (Sodium, OpenSSL, BCMath, ZipArchive).

Associated files

```
Frontend / View:
- includes/dashboard/views/view-kernel.php
- includes/dashboard/views/kernel/script.js

Backend:
- includes/modules/kernel/class-vis-kernel-sentinel.php

Styles:
- includes/dashboard/views/kernel/style.css
```

Dashboard integration

- **Pages:** Dashboard -> kernel
- **Components:** kernel status display, memory-usage graph, system parameters.
- **Styles:** kernel/style.css
- **Script:** kernel/script.js
- **Backend functions:** VIS_Kernel_Sentinel::get_system_report()

Dependencies

- Requires core: VIS_Security_Health

Used by

- VIS_Bootstrapper::engage_phase_2()

Subsystem: TRINITY XDR Fabric

Purpose

Extended Detection & Response (XDR) security fabric: unifies the event streams from all sensors (Cerberus, Aegis, Zeus, Prometheus, Nemesis, Morpheus), correlates individual attacks into coherent incidents, generates understandable attack stories and orchestrates reversible countermeasures with audit-safe Merkle evidence.

Capabilities

- Typed security-event model (XdrEvent) with severity, attribution score, actor hash and privacy masking.
- Cause-oriented incident correlation (IncidentEngine) across IP, session and time horizons.
- Rule-based policy evaluation (PolicyEngine) for automated responses.
- Reversible response orchestration (ResponseEngine): IP bans, session termination and quarantine with full rollback support.
- Merkle evidence chain (EvidenceStore): cryptographically linked digest history for each incident in wp_vis_xdr_evidence.

Associated files

```
Backend:
- includes/xdr/class-xdr-event-fabric.php
- includes/xdr/class-xdr-event.php
- includes/xdr/class-xdr-event-repository.php
- includes/xdr/class-xdr-incident-engine.php
- includes/xdr/class-xdr-policy-engine.php
- includes/xdr/class-xdr-response-engine.php
- includes/xdr/class-xdr-evidence-store.php
- includes/xdr/class-xdr-redactor.php
- includes/xdr/class-xdr-request-context.php

Dashboard / Frontend:
- includes/dashboard/views/view-trinity.php
- includes/dashboard/views/view-xdr.php
- includes/dashboard/views/view-thread.php
- includes/dashboard/views/trinity/style.css
- includes/dashboard/views/thread/style.css
- includes/dashboard/views/xdr/style.css
- assets/css/vis-xdr.css
- includes/dashboard/views/thread/script.js

Database:
- wp_vis_xdr_events
- wp_vis_xdr_incidents
- wp_vis_xdr_incident_events
- wp_vis_xdr_responses
- wp_vis_xdr_evidence

Tests:
- scripts/xdr-regression.php
- scripts/xdr-adversarial-regression.php
- scripts/xdr-release-blockers-regression.php
```

Dashboard integration

- **Pages:** Dashboard -> trinity, thread, xdr
- **Components:** live situation view, incident cards with attack story, response history with rollback button, cryptographic evidence certificates.
- **Data:** structured events, threat clusters, automated countermeasures.

Subsystem: Scanner & Malware Engine

Purpose

Provides continuous, paginated and resource-bounded file-integrity and malware scanning across the entire WordPress filesystem with SHA-256 Merkle-tree baselines and a secure quarantine vault.

Capabilities

- Resumable scans (VIS_Scanner_Engine_Omega) using NDJSON state files in the protected Vault.
- File detection through AST lexical flow, polyglot detection and SVG/XML sanitization.
- Quarantine of infected files in VIS_VAULT_DIR/quarantine/ with AES-256-GCM encryption.

Associated files

```
Backend:
- includes/scanner/class-vis-scanner-engine.php
- includes/scanner/class-vis-malware-engine.php
- includes/scanner/detectors/class-vis-php-lexical-detector.php
- includes/scanner/detectors/class-vis-polyglot-detector.php
- includes/scanner/detectors/class-vis-svg-xml-detector.php
- includes/scanner/detectors/class-vis-archive-detector.php
- includes/scanner/detectors/class-vis-path-context-detector.php
- includes/scanner/storage/class-vis-quarantine-store.php
- includes/scanner/contracts/interface-vis-file-detector.php
- includes/scanner/value/class-vis-scan-budget.php
- includes/scanner/value/class-vis-scan-context.php
- includes/scanner/value/class-vis-scan-finding.php
- includes/scanner/value/class-vis-scan-verdict.php

Frontend:
- assets/js/vis-scanner-client.js
- assets/css/vis-scanner-modal.css
- includes/dashboard/views/view-integrity.php
- includes/dashboard/views/integrity/script.js
- includes/dashboard/views/integrity/style.css

API:
- wp_ajax_vis_run_scan
- wp_ajax_vgt_integrity_uplink
- wp_ajax_vis_inspect_file
- wp_ajax_vis_approve_changes
```

Tests:

- scripts/scanner-resumption-regression.php
- scripts/integrity-baseline-regression.php
- scripts/malware-scanner-regression.php

Dynamic Add-On: Vision Legal Pro (VLP)

Purpose

Privacy and compliance extension: enables local asset mirroring (Google Fonts, JavaScripts) through the Shadow-Net Proxy, cookie-consent management and AI-assisted multilingual functionality (Lingua AI).

Capabilities

- Shadow-Net: intercepts external resource requests and mirrors assets locally without data leakage to US servers.
- Privacy Shield: manages cookie consent and blocks trackers before consent.
- Lingua AI: automatically translates website content through Groq LLM.

Associated files

Backend:

- includes/VLP/vision-legal-pro.php
- includes/VLP/includes/core/class-vlp-asset-library.php
- includes/VLP/includes/core/class-vlp-service-definitions.php
- includes/VLP/includes/admin/class-vlp-admin-dashboard.php
- includes/VLP/includes/modules/shadow-net/
- includes/VLP/includes/modules/privacy-shield/
- includes/VLP/includes/modules/lingua/
- includes/VLP/includes/modules/dattrack/

Dashboard:

- includes/dashboard/views/view-vlp.php
- includes/dashboard/views/vlp/style.css
- includes/dashboard/views/vlp/script.js

API:

- admin_post_vgt_sync
- admin_post_vgt_export_csv
- admin_post_vgt_export_pdf
- wp_ajax_vlp_save_banner
- wp_ajax_vlp_download_asset_secure

Dynamic Add-On: VisionGaia SEO Architect

Purpose

Full SEO suite: provides automated metadata generation, XML sitemaps, 301 redirect management, local Schema.org geo injection and relevance analysis.

Capabilities

- Automatic metatag generation through AI.
- High-performance sitemap generator without third-party plugins.
- Integrated 301/302 redirect manager with loop detection.

Associated files

Backend:

- includes/VisionGaiaSEO/visiongaia-seo-architect.php
- includes/VisionGaiaSEO/class-vg-seo-bootstrapper.php
- includes/VisionGaiaSEO/class-vg-seo-ajax-router.php
- includes/VisionGaiaSEO/class-vg-seo-settings.php
- includes/VisionGaiaSEO/includes/class-vg-api-service.php
- includes/VisionGaiaSEO/includes/class-vg-automation-engine.php
- includes/VisionGaiaSEO/includes/class-vg-geo-injector.php
- includes/VisionGaiaSEO/includes/class-vg-seo-relevance.php
- includes/VisionGaiaSEO/includes/class-vg-vault-bridge.php
- includes/VisionGaiaSEO/includes/class-vgt-data-manager.php
- includes/VisionGaiaSEO/includes/class-vgt-frontend-renderer.php
- includes/VisionGaiaSEO/includes/class-vgt-meta.php
- includes/VisionGaiaSEO/includes/class-vgt-redirect.php
- includes/VisionGaiaSEO/includes/class-vgt-sitemap.php

API:

- wp_ajax_vg_seo_generate
- wp_ajax_vg_seo_reset_stats
- wp_ajax_vg_seo_get_all_pending
- wp_ajax_vg_seo_get_all_completed

```
- wp_ajax_vgt_add_redirect
- wp_ajax_vgt_delete_redirect
- admin_post_vgt_vault_save
- admin_post_vgt_vault_delete

Tests:
- scripts/seo-relevance-regression.php
```

Dynamic Add-On: VisionGaia Builder

Purpose

Integrated visual editor and migrator: enables conversion of Elementor pages into lean native GeDefense code as well as visual live editing with an AI Copilot.

Capabilities

- Bulk migration of Elementor posts into standards-compliant HTML/CSS.
- Isolated live-preview broker for safe rendering.
- AI Copilot for generating responsive layouts.

Associated files

```
Backend:
- includes/builder/builder.php
- includes/builder/inc/class-vgt-admin.php
- includes/builder/inc/class-vgt-ajax.php
- includes/builder/inc/class-vgt-content-gate.php
- includes/builder/inc/class-vgt-frontend.php
- includes/builder/inc/class-vgt-migration.php
- includes/builder/inc/class-vgt-preview-broker.php
- includes/builder/inc/class-vgt-telemetry.php

Views:
- includes/builder/views/dashboard.php
- includes/builder/views/editor-ui.php
- includes/builder/views/bulk-migration.php

API:
- wp_ajax_vgt_render_preview
- wp_ajax_vgt_copilot_generate
- wp_ajax_vgt_migrate_elementor
- wp_ajax_vgt_get_elementor_pages
- wp_ajax_vgt_process_bulk_migration

Tests:
- scripts/builder-live-preview-regression.php
```

4. DASHBOARD - DETAILED DOCUMENTATION

In the GeDefense Cockpit, every page is routed through the central menu page `admin.php?page=vgt-suite&tab={tab}`. Rendering is handled by `VIS_Dashboard_View::render()`, while asset loading is controlled by `VIS_Dashboard_Assets::enqueue()`.

Dashboard -> Overview (Control Center)

- **Route:** `admin.php?page=vgt-suite&tab=overview`
- **Frontend file:** `includes/dashboard/views/view-overview.php`
- **CSS / Styling:** `includes/dashboard/views/overview/style.css`, `assets/css/vis-dashboard.css`, `assets/css/vis-dashboard-modern.css`
- **Components used:** Vital HUD with score gauges, system quick-check matrix, live deny-list excerpt, scan-trigger button.
- **Responsible module:** Core Cockpit / Health Engine (`VIS_Security_Health`, `VIS_Security_Center`)
- **API calls:** `wp_ajax_vis_run_scan`, `wp_ajax_vis_dashboard_unban_ip`
- **Backend handlers:** `VIS_Dashboard_Ajax::handle_scan_bridge()`, `VIS_Dashboard_Ajax::handle_unban_ip()`
- **Services:** `VIS_Security_Health::evaluate()`, `VIS_Scanner_Engine_Omega`
- **Database access:** reads `wp_vis_apex_bans`, `wp_vis_omega_logs`, `wp_vis_xdr_incidents`
- **Authentication/permissions:** `current_user_can('manage_options')`, nonce: `vis_nonce`
- **Data flow:**

```

User opens Overview
↓
view-overview.php loads vital status & log summary
↓
GET from wp_vis_apex_bans, wp_vis_omega_logs, wp_vis_xdr_incidents
↓
HTML rendering (server-side)
↓
[Optional] User clicks "Check system integrity"
↓
overview/script.js sends AJAX 'vis_run_scan'
↓
VIS_Dashboard_Ajax::handle_scan_bridge()
↓
VIS_Scanner_Engine_Omega::run_batch()
↓
JSON response with progress & findings
↓
UI update in the scan modal

```

Dashboard -> Thread (Threat Matrix)

- **Route:** admin.php?page=vgt-suite&tab=thread
- **Frontend file:** includes/dashboard/views/view-thread.php
- **CSS / Styling:** includes/dashboard/views/thread/style.css
- **Components used:** XDR event feed, filter bar (severity, module, IP), detail inspector.
- **Responsible module:** TRINITY XDR / Prometheus
- **API calls:** client-side DOM filtering through thread/script.js
- **Backend handler:** server-side rendering through PHP
- **Database access:** reads wp_vis_omega_logs, wp_vis_xdr_events
- **Authentication/permissions:** manage_options

Dashboard -> Oracle (Oracle Scanner)

- **Route:** admin.php?page=vgt-suite&tab=oracle
- **Frontend file:** includes/dashboard/views/view-oracle.php
- **CSS / Styling:** includes/dashboard/views/oracle/style.css
- **Components used:** 12-vector audit matrix, anomaly radar, remediation guide.
- **Responsible module:** Oracle (VIS_Oracle)
- **API calls:** wp_ajax_vis_oracle_ping
- **Backend handler:** VIS_Dashboard_Ajax::handle_oracle_ping()
- **Services:** VIS_Oracle::get_audit_findings()
- **Database access:** reads/writes wp_vis_oracle_patterns
- **Authentication/permissions:** manage_options, nonce: vis_oracle_ping

Dashboard -> Integrity (Integrity Monitor)

- **Route:** admin.php?page=vgt-suite&tab=integrity
- **Frontend file:** includes/dashboard/views/view-integrity.php
- **CSS / Styling:** includes/dashboard/views/integrity/style.css
- **Components used:** Merkle-tree hash status display, file-explorer modal with syntax highlighting, approval trigger for modifications.
- **Responsible module:** Core Module Integrity (VIS_Module_Integrity) & Scanner Engine
- **API calls:** wp_ajax_vis_inspect_file, wp_ajax_vis_approve_changes, wp_ajax_vis_run_scan
- **Backend handlers:** VIS_Dashboard_Ajax::handle_inspect_file(), VIS_Dashboard_Ajax::handle_approve()
- **Services:** VIS_Module_Integrity::verify_all(), VIS_Quarantine_Store
- **Database access:** options: vis_file_hashes, file integrity/module-manifest.json
- **Authentication/permissions:** manage_options, nonce: vis_nonce

Dashboard -> Security Center

- **Route:** admin.php?page=vgt-suite&tab=security_center (&security_section=assurance|system|logs)

- **Frontend file:** `includes/dashboard/views/view-security_center.php` (embeds `view-systatus.php` and `view-logs.php`)
 - **CSS / Styling:** `assets/css/vis-security-center.css`, `systatus/style.css`, `logs/style.css`
 - **Components used:** sub-navigation, self-test trigger, system-diagnostics metrics, searchable/filterable logbook.
 - **Responsible module:** Security Center (`VIS_Security_Center`), Security Health (`VIS_Security_Health`), Event Bus
 - **API calls:** `wp_ajax_vis_security_center_test`, `wp_ajax_vis_dashboard_unban_ip`
 - **Backend handlers:** `VIS_Dashboard_Ajax::handle_security_center_test()`, `VIS_Dashboard_Ajax::handle_unban_ip()`
 - **Services:** `VIS_Security_Center::run_diagnostics()`, `VIS_Security_Health::evaluate()`
 - **Database access:** reads `wp_vis_omega_logs`, `wp_vis_apex_bans`, `wp_vis_xdr_events`
 - **Authentication/permissions:** `manage_options`, `nonce: vis_nonce`
-

Dashboard -> Trinity (TRINITY XDR)

- **Route:** `admin.php?page=vgt-suite&tab=trinity`
 - **Frontend file:** `includes/dashboard/views/view-trinity.php`
 - **CSS / Styling:** `includes/dashboard/views/trinity/style.css`, `assets/css/vis-xdr.css`
 - **Components used:** four-pillar Interlock HUD (Cerberus, Aegis, Prometheus, Nemesis), vector-distribution chart, quick-configuration switches.
 - **Responsible module:** TRINITY XDR / Trinity Grid (`VIS_Trinity_Grid`)
 - **API calls:** form mutations through Settings
 - **Backend handler:** `VIS_Dashboard_Settings::process_mutations()`
 - **Database access:** reads `wp_vis_xdr_events`, `wp_vis_xdr_incidents`, `wp_vis_apex_bans`
 - **Authentication/permissions:** `manage_options`, `nonce: vis_save_config`
-

Dashboard -> Zeus (Zeus Defender)

- **Route:** `admin.php?page=vgt-suite&tab=zeus`
 - **Frontend file:** `includes/dashboard/views/view-zeus.php`
 - **CSS / Styling:** `includes/dashboard/views/zeus/style.css`
 - **Components used:** WAF rule matrix, latency benchmark terminal, Blackbox drain monitor, Admission Token Generator, preset selector.
 - **Responsible module:** Zeus (`VIS_Zeus`)
 - **API calls:** 9 AJAX endpoints (`vis_save_zeus_config`, `vis_zeus_run_benchmark`, etc.)
 - **Backend handler:** `VIS_Dashboard_Ajax::handle_zeus_*`
 - **Services:** `Zeus_Compiler`, `Zeus_Benchmark`, `Zeus_Admission`
 - **Database access:** options: `vis_zeus_config`, `vis_zeus_contracts`
 - **Authentication/permissions:** `manage_options`, `nonce: vis_nonce`
-

Dashboard -> Aegis (AEGIS Firewall)

- **Route:** `admin.php?page=vgt-suite&tab=aegis`
 - **Frontend file:** `includes/dashboard/views/view-aegis.php`
 - **CSS / Styling:** `includes/dashboard/views/aegis/style.css`
 - **Components used:** mode selector (Monitor/Enforce), whitelist text areas for IPs and user agents, diagnostics terminal.
 - **Responsible module:** Aegis (`VIS_Aegis`)
 - **API calls:** `wp_ajax_vis_oracle_ping` (through `vis-oracle-diagnostics.js`)
 - **Backend handlers:** `VIS_Dashboard_Settings::process_mutations()`, `VIS_Dashboard_Ajax::handle_oracle_ping()`
 - **Database access:** options: `vis_config` (keys: `aegis_enabled`, `aegis_mode`, `aegis_whitelist_ips`, `aegis_whitelist_uas`)
 - **Authentication/permissions:** `manage_options`, `nonce: vis_save_config`
-

Dashboard -> Prometheus (Prometheus Engine)

- **Route:** `admin.php?page=vgt-suite&tab=prometheus`
- **Frontend file:** `includes/dashboard/views/view-prometheus.php`

- **CSS / Styling:** `includes/dashboard/views/prometheus/style.css`
 - **Components used:** subnet-risk heatmap, threshold slider, IP whitelist.
 - **Responsible module:** Prometheus (VIS_Prometheus)
 - **API calls:** client filter through `prometheus/script.js`, form storage through Settings
 - **Database access:** options: `vis_config` (Prometheus keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Cerberus (Cerberus IP Ban)

- **Route:** `admin.php?page=vgt-suite&tab=cerberus`
 - **Frontend file:** `includes/dashboard/views/view-cerberus.php`
 - **CSS / Styling:** `includes/dashboard/views/cerberus/style.css`
 - **Components used:** paginated live deny list, unban action, manual ban form, server-rule download.
 - **Responsible module:** Cerberus (VIS_Cerberus)
 - **API calls:** `wp_ajax_vis_dashboard_unban_ip`
 - **Backend handler:** `VIS_Dashboard_Ajax::handle_unban_ip()`
 - **Services:** `VIS_Cerberus::instance()->unban_ip()`
 - **Database access:** reads/deletes `wp_vis_apex_bans`
 - **Authentication/permissions:** `manage_options`, nonce: `vis_nonce`
-

Dashboard -> Airlock

- **Route:** `admin.php?page=vgt-suite&tab=airlock`
 - **Frontend file:** `includes/dashboard/views/view-airlock.php`
 - **CSS / Styling:** `includes/dashboard/views/airlock/style.css`
 - **Components used:** file-inspection status cards, SVG sanitization toggle, quarantine options.
 - **Responsible module:** Airlock (VIS_Airlock)
 - **API calls:** form mutations through Settings
 - **Database access:** options: `vis_config` (Airlock keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Nemesis (Nemesis Deception)

- **Route:** `admin.php?page=vgt-suite&tab=nemesis`
 - **Frontend file:** `includes/dashboard/views/view-nemesis.php`
 - **CSS / Styling:** `includes/dashboard/views/nemesis/style.css`
 - **Components used:** tarpit-latency configurator, fake-response selector, honeypot audit.
 - **Responsible module:** Nemesis (VIS_Nemesis)
 - **API calls:** Settings mutation
 - **Database access:** options: `vis_config` (Nemesis keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Ghost Trap (Ghost Honeypot)

- **Route:** `admin.php?page=vgt-suite&tab=ghost_trap`
 - **Frontend file:** `includes/dashboard/views/view-ghost_trap.php`
 - **CSS / Styling:** `includes/dashboard/views/ghost_trap/style.css`
 - **Components used:** list of active decoy files (`.env`, `wp-config.php.bak`), hit statistics.
 - **Responsible module:** Ghost Trap (VIS_Ghost_Trap)
 - **API calls:** Settings mutation
 - **Database access:** options: `vis_config` (keys: `ghost_trap_enabled`, `ghost_trap_exts`)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Hades (Hades Stealth)

- **Route:** `admin.php?page=vgt-suite&tab=hades`
 - **Frontend file:** `includes/dashboard/views/view-hades.php`
 - **CSS / Styling:** `includes/dashboard/views/hades/style.css`
 - **Components used:** secret login URL generator, Nginx/Apache rewrite-rule box.
 - **Responsible module:** Hades (VIS_Hades)
 - **API calls:** Settings mutation
 - **Database access:** options: `vis_config` (Hades keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Morpheus (Morpheus Sandbox)

- **Route:** `admin.php?page=vgt-suite&tab=morpheus`
 - **Frontend file:** `includes/dashboard/views/view-morpheus.php`
 - **CSS / Styling:** `includes/dashboard/views/morpheus/style.css`
 - **Components used:** live call-stack inspector, Hook Audit table with allow/block actions, AI stack analysis.
 - **Responsible module:** Morpheus (Vis_Morpheus)
 - **API calls:** 5 AJAX endpoints (`vgt_morpheus_trigger_ai`, etc.)
 - **Backend handler:** `Morpheus_Dashboard::ajax_*`
 - **Database access:** options: `vgt_morpheus_audit_matrix`, `vis_config`
 - **Authentication/permissions:** `manage_options`, nonce: `vis_nonce` / `vgt_morpheus_isolation`
-

Dashboard -> Titan (Titan Hardening)

- **Route:** `admin.php?page=vgt-suite&tab=titan`
 - **Frontend file:** `includes/dashboard/views/view-titan.php`
 - **CSS / Styling:** `assets/css/vis-titan.css`
 - **Components used:** 5-stage Policy Lifecycle console, CSP directive editor, violation viewer, server-config download.
 - **Responsible module:** Titan (VIS_Titan)
 - **API calls:** `admin_post_vis_titan_policy_action`, `admin_post_vis_titan_download_nginx`, `admin_post_vis_titan_generate_gate_link`
 - **Backend handlers:** `VIS_Titan::handle_policy_action()`, `VIS_Titan_Server_Rules::handleDownload()`
 - **Database access:** options: `vis_config` (Titan keys), `vis_titan_policies`, `vis_titan_violations`
 - **Authentication/permissions:** `manage_options`, nonce: `vis_titan_policy_action`
-

Dashboard -> Kernel (Kernel Uplink)

- **Route:** `admin.php?page=vgt-suite&tab=kernel`
 - **Frontend file:** `includes/dashboard/views/view-kernel.php`
 - **CSS / Styling:** `includes/dashboard/views/kernel/style.css`
 - **Components used:** low-level system parameters, OPcache status, memory-peak gauge.
 - **Responsible module:** Kernel Sentinel (VIS_Kernel_Sentinel)
 - **Database access:** no direct DB writes
 - **Authentication/permissions:** `manage_options`
-

Dashboard -> Styx (Styx Controller)

- **Route:** `admin.php?page=vgt-suite&tab=styx`
- **Frontend file:** `includes/dashboard/views/view-styx.php`
- **CSS / Styling:** `includes/dashboard/views/styx/style.css`
- **Components used:** outbound-traffic log, whitelist-domain editor.
- **Responsible module:** Styx (VIS_Styx)
- **API calls:** Settings mutation
- **Database access:** options: `vis_config` (Styx keys)
- **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`

Dashboard -> Chronos (Chronos Autopilot)

- **Route:** `admin.php?page=vgt-suite&tab=chronos`
 - **Frontend file:** `includes/dashboard/views/view-chronos.php`
 - **CSS / Styling:** `includes/dashboard/views/chronos/style.css`
 - **Components used:** WP-Cron scheduler status, frequency selector (hourly/daily), email alert editor.
 - **Responsible module:** Chronos (VIS_Chronos)
 - **API calls:** Settings mutation
 - **Database access:** options: `vis_config` (Chronos keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> VLP (Privacy & Shadow-Net)

- **Route:** `admin.php?page=vgt-suite&tab=vlp`
 - **Frontend file:** `includes/dashboard/views/view-vlp.php`
 - **CSS / Styling:** `includes/dashboard/views/vlp/style.css`
 - **Components used:** Shadow-Net Proxy status, local asset directory, GDPR Shield settings.
 - **Responsible module:** Vision Legal Pro Bridge (VIS_Bridge_Vision_Legal_Pro) / VLP
 - **API calls:** `admin_post_vgt_sync`, `wp_ajax_vlp_download_asset_secure`
 - **Database access:** options: `vis_vlp_config`
 - **Authentication/permissions:** `manage_options`
-

Dashboard -> Filesystem (Data Security)

- **Route:** `admin.php?page=vgt-suite&tab=filesystem`
 - **Frontend file:** `includes/dashboard/views/view-filesystem.php`
 - **CSS / Styling:** `includes/dashboard/views/filesystem/style.css`
 - **Components used:** CHMOD permission table for core files with traffic-light status colors.
 - **Responsible module:** Filesystem Guard (VIS_Filesystem_Guard)
 - **Backend handler:** server-side rendering
 - **Database access:** none (checks filesystem directly)
 - **Authentication/permissions:** `manage_options`
-

Dashboard -> Vault (Key Vault)

- **Route:** `admin.php?page=vgt-suite&tab=vault`
 - **Frontend file:** `includes/dashboard/views/view-vault.php`
 - **CSS / Styling:** `includes/dashboard/views/vault/style.css`
 - **Components used:** secret input (Key ID & password), registry table of encrypted keys with delete function.
 - **Responsible module:** Key Vault (VIS_Key_Vault)
 - **API calls:** post actions: `vis_vault_save`, `vis_vault_delete`
 - **Backend handler:** `VIS_Key_Vault::handle_post()`
 - **Database access:** options: `vis_key_vault_registry`, encrypted blobs
 - **Authentication/permissions:** `manage_options`, nonces: `vis_vault_save_action`, `vis_vault_delete_action`
-

Dashboard -> ThroneGuard

- **Route:** `admin.php?page=vgt-throneguard` or `admin.php?page=vgt-suite&tab=throneguard`
- **Frontend file:** `includes/dashboard/views/view-throneguard.php`
- **CSS / Styling:** `includes/dashboard/views/throneguard/style.css`
- **Components used:** Master Admin Claiming card, role-permission table, Superkey session unlocker, audit log.
- **Responsible module:** ThroneGuard (VIS_Throne_Guard)
- **API calls:** `admin_post_vis_throneguard_claim`, `admin_post_vis_throneguard_save`, `admin_post_vis_throneguard_unlock`, `wp_ajax_vis_throneguard_clear_logs`

- **Backend handler:** `VIS_Throne_Guard::handle_*`
 - **Database access:** options: `vis_throneguard_config`, `vis_throneguard_audit_log`
 - **Authentication/permissions:** `manage_options`, nonces: `vis_throneguard_nonce`, `vis_throneguard_claim_nonce`, `vis_throneguard_unlock_nonce`
-

Dashboard -> LoginPager

- **Route:** `admin.php?page=vgt-loginpager` or `admin.php?page=vgt-suite&tab=loginpager`
 - **Frontend file:** `includes/dashboard/views/view-loginpager.php`
 - **CSS / Styling:** `includes/dashboard/views/loginpager/style.css`
 - **Components used:** dual-column studio with interactive live preview of the login page, preset swatches, color picker with hex input, image URL binding.
 - **Responsible module:** LoginPager (`VIS_LoginPager`)
 - **API calls:** Settings mutation through form
 - **Backend handler:** `VIS_Dashboard_Settings::process_mutations()`
 - **Database access:** options: `vis_config` (LoginPager keys)
 - **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`
-

Dashboard -> Downloads (Secure Downloads)

- **Route:** `admin.php?page=vgt-suite&tab=downloads`
 - **Frontend file:** `includes/dashboard/views/view-downloads.php`
 - **CSS / Styling:** `includes/dashboard/views/downloads/style.css`
 - **Components used:** Media Library selector for link generation, table of active downloads with click-to-copy URL and counter.
 - **Responsible module:** Secure Downloads (`DownloadManager`)
 - **API calls:** `admin_post_vis_download_register`, `admin_post_vis_download_toggle`, `admin_post_vis_download_delete`
 - **Backend handlers:** `DownloadManager::registerAction()`, `toggleAction()`, `deleteAction()`
 - **Database access:** reads/writes `wp_vis_secure_downloads`
 - **Authentication/permissions:** `manage_options`, nonces: `vis_download_register`, `vis_download_toggle`, `vis_download_delete`
-

Dashboard -> Modules (Add-On Management)

- **Route:** `admin.php?page=vgt-suite&tab=modules`
 - **Frontend file:** `includes/dashboard/views/view-modules.php`
 - **CSS / Styling:** `includes/dashboard/views/modules/style.css`
 - **Components used:** drag-and-drop ZIP uploader, grid of installed Add-Ons (VLP, Builder, SEO) with uninstall buttons.
 - **Responsible module:** Module Registry (`VIS_Module_Registry`)
 - **API calls:** `wp_ajax_vis_upload_addon`, `wp_ajax_vis_uninstall_addon`
 - **Backend handlers:** `VIS_Dashboard_Ajax::handle_upload_addon()`, `VIS_Dashboard_Ajax::handle_uninstall_addon()`
 - **Services:** `VIS_Module_Registry::all()`, ZipArchive security checks (zip-bomb & traversal protection)
 - **Database access:** options: `vis_config`
 - **Authentication/permissions:** `manage_options`, nonce: `vis_nonce`
-

Dashboard -> Setup Wizard

- **Route:** `admin.php?page=vgt-suite&tab=setup_wizard`
- **Frontend file:** `includes/dashboard/views/view-setup_wizard.php`
- **CSS / Styling:** `includes/dashboard/views/setup_wizard/style.css`
- **Components used:** seven-step assistant (administrator IP whitelist, protection level, WAF, RASP, deception, hardening, completion).
- **Responsible module:** Dashboard Setup Flow
- **API calls:** Settings mutation with automatic completion (`vgt_setup_wizard_completed = 1`)
- **Backend handler:** `VIS_Dashboard_Settings::process_mutations()`

- **Services:** `VIS_Security::client_ip()`
- **Database access:** options: `vis_config`, `vgt_setup_wizard_completed`
- **Authentication/permissions:** `manage_options`, nonce: `vis_save_config`

Dashboard -> XDR (Situation View & Incident Deep Dive)

- **Route:** `admin.php?page=vgt-suite&tab=xdr`
- **Frontend file:** `includes/dashboard/views/view-xdr.php`
- **CSS / Styling:** `includes/dashboard/views/xdr/style.css`, `assets/css/vis-xdr.css`
- **Components used:** detailed situation view, attack stories, causal event graph, response rollback trigger, Evidence Root Inspector.
- **Responsible module:** TRINITY XDR (`VisionGaia\GeDefense\Xdr\*`)
- **Database access:** `wp_vis_xdr_incidents`, `wp_vis_xdr_events`, `wp_vis_xdr_responses`, `wp_vis_xdr_evidence`
- **Authentication/permissions:** `manage_options`

Dashboard -> Gorgon (Neural Sync Matrix)

- **Route:** `admin.php?page=vgt-suite&tab=gorgon`
- **Frontend file:** `includes/dashboard/views/view-gorgon.php`
- **CSS / Styling:** `includes/dashboard/views/gorgon/style.css`
- **Components used:** mesh-node status, synchronization log, peer configurator.
- **Responsible module:** Gorgon (`VisionGaia\GeDefense\Modules\Gorgon\Gorgon`)
- **API calls:** 6 AJAX routes (`vgt_gorgon_*`)
- **Backend handler:** `Gorgon_Ajax::handle_*`
- **Database access:** options: `vis_config` (Gorgon keys)
- **Authentication/permissions:** `manage_options`, nonce: `vgt_gorgon_nonce`

5. CSS / UI ARCHITECTURE

The entire GeDefense UI design system is based on a **glassmorphism dark-mode matrix** using native CSS variables (`Design Tokens`). There are **no external CSS frameworks or webfonts**.

5.1 Global Design Tokens & Variables

Defined in `assets/css/vis-dashboard.css` and refined in `assets/css/vis-dashboard-modern.css`:

```
:root {
  --vgt-bg-base: #070a13;
  --vgt-bg-panel: rgba(13, 18, 36, 0.7);
  --vgt-bg-panel-elevated: rgba(20, 27, 54, 0.85);
  --vgt-border: rgba(255, 255, 255, 0.08);
  --vgt-border-hover: rgba(255, 255, 255, 0.2);
  --vgt-text-main: #f8fafc;
  --vgt-text-dim: #94a3b8;
  --vgt-text-muted: #64748b;
  --vgt-neon-purple: #a855f7;
  --vgt-neon-green: #10b981;
  --vgt-neon-red: #ef4444;
  --vgt-neon-blue: #3b82f6;
  --vgt-neon-cyan: #06b6d4;
  --vgt-neon-gold: #f59e0b;
}
```

5.2 Stylesheet Mapping Matrix

```
assets/css/vis-dashboard.css
└─ Affects: entire GeDefense Dashboard (base layout, topbar, sidebar, panel grid)
└─ Loaded on: all pages under 'admin.php?page=vgt-suite', 'vgt-throneguard', 'vgt-loginpager'

assets/css/vis-dashboard-modern.css
└─ Affects: responsive breakpoints (@media (max-width: 960px)), accessibility (:focus-visible), prefers-reduced-motion
└─ Loaded on: all dashboard pages

assets/css/vis-scanner-modal.css
└─ Affects: scan-progress modal and code-inspector explorer
└─ Loaded on: all dashboard pages (for the global scanner client)

assets/css/vis-security-center.css
```

```
└─ Affects: Dashboard -> Security Center (tabs: Assurance, System Status, Logs)
└─ Loaded on: tab 'security_center'
```

```
assets/css/vis-titan.css
```

```
└─ Affects: Dashboard -> Titan Hardening
└─ Loaded on: tab 'titan'
```

```
assets/css/vis-xdr.css
```

```
└─ Affects: Dashboard -> Trinity, Thread, XDR
└─ Loaded on: tabs 'trinity', 'thread', 'xdr'
```

```
includes/dashboard/views/sidebar/style.css
```

```
└─ Affects: left navigation bar, SVG icons, section dividers
└─ Loaded on: all dashboard pages
```

```
includes/dashboard/views/{tab}/style.css (27 module-specific stylesheets)
```

```
└─ aegis/style.css      -> Tab 'aegis'
└─ airlock/style.css   -> Tab 'airlock'
└─ cerberus/style.css  -> Tab 'cerberus'
└─ chronos/style.css   -> Tab 'chronos'
└─ downloads/style.css -> Tab 'downloads'
└─ filesystem/style.css -> Tab 'filesystem'
└─ ghost_trap/style.css -> Tab 'ghost_trap'
└─ gorgon/style.css    -> Tab 'gorgon'
└─ hades/style.css     -> Tab 'hades'
└─ integrity/style.css -> Tab 'integrity'
└─ kernel/style.css    -> Tab 'kernel'
└─ loginpager/style.css -> Tab 'loginpager' & 'vgt-loginpager'
└─ logs/style.css      -> Tab 'security_center' (Logs section)
└─ modules/style.css   -> Tab 'modules'
└─ morpheus/style.css  -> Tab 'morpheus'
└─ nemesis/style.css   -> Tab 'nemesis'
└─ oracle/style.css    -> Tab 'oracle'
└─ overview/style.css  -> Tab 'overview'
└─ prometheus/style.css -> Tab 'prometheus'
└─ setup_wizard/style.css -> Tab 'setup_wizard'
└─ styx/style.css      -> Tab 'styx'
└─ systatus/style.css  -> Tab 'security_center' (System section)
└─ thread/style.css    -> Tab 'thread'
└─ throneguard/style.css -> Tab 'throneguard' & 'vgt-throneguard'
└─ trinity/style.css   -> Tab 'trinity'
└─ vault/style.css     -> Tab 'vault'
└─ vlp/style.css       -> Tab 'vlp'
└─ xdr/style.css       -> Tab 'xdr'
└─ zeus/style.css      -> Tab 'zeus'
```

6. API ARCHITECTURE

6.1 AJAX Endpoints (wp_ajax_*)

Action Name	Method	Frontend Caller	Handler	Service / Class	Data Source / Action
vis_approve_changes	POST	integrity/script.js	handle_approve	VIS_Dashboard_Ajax	Updates manifest baseline
vis_save_zeus_config	POST	zeus/script.js	handle_zeus_config	Zeus_Config_Repository	Stores WAF configuration
vis_zeus_run_benchmark	POST	zeus/script.js	handle_zeus_benchmark	Zeus_Benchmark	Runs 1000 WAF RegEx iterations
vis_zeus_run_self_test	POST	zeus/script.js	handle_zeus_self_test	Zeus_Contracts	Validates WAF signatures
vis_zeus_drain_blackbox	POST	zeus/script.js	handle_zeus_drain_blackbox	Zeus_Blackbox	Drains WAF telemetry buffer
vis_zeus_restore_preset	POST	zeus/script.js	handle_zeus_restore_preset	Zeus_Config_Repository	Restores WAF defaults
vis_zeus_save_contract	POST	zeus/script.js	handle_zeus_save_contract	Zeus_Contracts	Stores WAF rule contract
vis_zeus_delete_contract	POST	zeus/script.js	handle_zeus_delete_contract	Zeus_Contracts	Deletes WAF rule contract
vis_zeus_generate_admission_token	POST	zeus/script.js	handle_zeus_generate_token	Zeus_Admission	Generates signed WAF bypass token
vis_zeus_rollback_policy	POST	zeus/script.js	handle_zeus_rollback_policy	Zeus_Config_Repository	Rolls back to previous policy

Action Name	Method	Frontend Caller	Handler	Service / Class	Data Source / Action
vis_dashboard_unban_ip	POST	cerberus/script.js, vis-dashboard.js	handle_unban_ip	VIS_Cerberus	Deletes IP from wp_vis_apex_bans
vis_run_scan	POST	vis-scanner-client.js	handle_scan_bridge	VIS_Scanner_Engine_Omega	Executes batch file scan
vgt_integrity_upload	POST	vis-scanner-client.js	handle_scan_bridge	VIS_Scanner_Engine_Omega	Alias for scan batch
vis_inspect_file	POST	integrity/script.js	handle_inspect_file	VIS_Dashboard_Ajax	Reads file source in jailed path
vis_security_center_test	POST	vis-security-center.js	handle_security_center_test	VIS_Security_Center	Runs host & kernel diagnostics
vis_oracle_ping	POST	vis-oracle-diagnostics.js	handle_oracle_ping	VIS_Oracle	Ping test against WAF & DB
vis_upload_addon	POST	modules/script.js	handle_upload_addon	VIS_Module_Registry	Validates & extracts ZIP Add-On
vis_uninstall_addon	POST	modules/script.js	handle_uninstall_addon	VIS_Module_Registry	Deletes dynamic Add-On directory
vgt_gorgon_toggle	POST	gorgon/script.js	handle_toggle	Gorgon_Ajax	Enables/disables mesh sync
vgt_gorgon_update_config	POST	gorgon/script.js	handle_update_config	Gorgon_Ajax	Stores Nexus URL & API key
vgt_gorgon_ping_nexus	POST	gorgon/script.js	handle_ping_nexus	Gorgon_Uplink	Pings remote Nexus server
vgt_gorgon_sync	POST	gorgon/script.js	handle_sync	Gorgon_Sync_Engine	Executes ad-hoc mesh synchronization
vgt_gorgon_add_node	POST	gorgon/script.js	handle_add_node	Gorgon_Config	Registers new peer node
vgt_gorgon_remove_node	POST	gorgon/script.js	handle_remove_node	Gorgon_Config	Removes peer node
vgt_morpheus_trigger_ai	POST	morpheus/script.js	ajax_trigger_ai	Morpheus_AI	Starts AI call-stack analysis
vgt_morpheus_reject_ai	POST	morpheus/script.js	ajax_reject_ai	Morpheus_Dashboard	Rejects AI proposal
vgt_morpheus_approve_ai	POST	morpheus/script.js	ajax_approve_ai	Morpheus_Dashboard	Applies AI rule
vgt_morpheus_delete_matrix	POST	morpheus/script.js	ajax_delete_matrix	Morpheus_Dashboard	Cleans Audit Matrix
vgt_morpheus_toggle_strict	POST	morpheus/script.js	ajax_toggle_strict	Morpheus_Dashboard	Toggles RASP Strict Mode
vis_throneguard_clear_logs	POST	throneguard/script.js	handle_clear_logs	VIS_Throne_Guard	Clears ThroneGuard audit log
vg_seo_generate	POST	VisionGaiaSEO UI	handle_generate_seo	VG_Automation_Engine	Generates SEO metadata through AI
vg_seo_reset_stats	POST	VisionGaiaSEO UI	handle_reset_stats	VG_SEO_Ajax_Router	Resets SEO counters
vgt_add_redirect	POST	VisionGaiaSEO UI	handle_add_redirect	VGT_Redirect	Creates 301 redirect
vgt_delete_redirect	POST	VisionGaiaSEO UI	handle_delete_redirect	VGT_Redirect	Deletes 301 redirect
vgt_render_preview	POST	builder/views/editor-ui.php	ajax_render_preview	VGT_Ajax	Renders live preview
vgt_copilot_generate	POST	builder/views/editor-ui.php	ajax_copilot_generate	VGT_Ajax	Generates UI component through AI

Action Name	Method	Frontend Caller	Handler	Service / Class	Data Source / Action
vgt_migrate_elementor	POST	builder/views/bulk-migration.php	ajax_migrate	VGT_Migration	Migrates Elementor post
vlp_download_asset_secure	POST	vlp/script.js	handle_ajax_download	VLP_Asset_Downloader	Downloads asset into local proxy

6.2 Admin-Post Actions (admin_post_*)

Action Name	Method	Caller	Handler	Class	Function
vgt_export_sentinel_data	POST	view-security_center.php	handle	VIS_Sentinel_Export	Generates pseudonymized JSON export
vis_download_register	POST	view-downloads.php	registerAction	DownloadManager	Creates protected file copy
vis_download_toggle	POST	view-downloads.php	toggleAction	DownloadManager	Pauses/enables download link
vis_download_delete	POST	view-downloads.php	deleteAction	DownloadManager	Deletes protected file copy
vis_throneguard_claim	POST	view-throneguard.php	handle_claim_master	VIS_Throne_Guard	Assigns Master Admin role
vis_throneguard_save	POST	view-throneguard.php	handle_save	VIS_Throne_Guard	Stores role permissions
vis_throneguard_unlock	POST	view-throneguard.php	handle_unlock	VIS_Throne_Guard	Validates Superkey & unlocks session
vis_titan_policy_action	POST	assets/js/vis-titan-command-center.js	handle_policy_action	VIS_Titan	Executes stage transition (Test/Enforce/Rollback)
vis_titan_download_nginx	POST	view-titan.php	handleDownload	VIS_Titan_Server_Rules	Generates Nginx configuration
vis_titan_generate_gate_link	POST	assets/js/vis-titan-command-center.js	handleGenerateLink	VIS_Titan_Login_Gate	Generates one-time login link
vis_titan_preview_link	POST	assets/js/vis-titan-command-center.js	handlePreviewLink	VIS_Titan_Sandbox	Generates secure sandbox preview URL
vgt_sync	POST	view-vlp.php	process_live_sync	VLP_Admin_Dashboard	Runs VLP asset synchronization
vgt_export_csv	POST	view-vlp.php	stream_csv	VLP_Admin_Dashboard	Exports consent logs as CSV
vgt_export_pdf	POST	view-vlp.php	render_print_view	VLP_Admin_Dashboard	Print view for compliance audits
vgt_vault_save	POST	VisionGaiaSE0 UI	handle_save	VG_Vault_Bridge	Stores API keys encrypted in Vault
vgt_vault_delete	POST	VisionGaiaSE0 UI	handle_delete	VG_Vault_Bridge	Deletes API keys from Vault

6.3 REST API Endpoints

Route	Method	Authentication	Handler	Class	Purpose
/wp-json/visiongaia/v1/titan/csp-report	POST	None (return_true)	collect	VIS_Titan_Violation_Collector	Receives standardized CSP violation reports from the browser

7. DATA FLOWS

7.1 Ingress Request Inspection & Attack Drop Pipeline

```

[ Incoming HTTP request ]
↓
[ Phase 1: Pre-Flight Kernel ]
↓
1. Cerberus: 0(1) Memory Lookup (APCu)
  └─ IP banned? -> HTTP 403 (termination in < 0.1 ms, 0% DB load)
  └─ IP not banned -> Continue
↓
2. Zeus: Pre-Boot 6G DFA Match
  └─ Malicious query string / bad bot? -> HTTP 403 drop & telemetry logging
  └─ Clean -> Continue
↓
3. Aegis: Deep Packet Inspection (DPI)
  └─ Normalization (SQL comments, Unicode, quote slashing)
  └─ Recursive token inspection (GET, POST, headers, JSON)
  └─ Exploit detected? -> HTTP 403 + event to EventBus + ban through Cerberus
  └─ Clean -> Continue
↓
4. Prometheus: Behavioral Scoring
  └─ Accumulate subnet/IP score & inspect webshell heuristics
  └─ Threshold exceeded? -> ban through Cerberus
  └─ Clean -> Continue
↓
5. Nemesis: Deception Check
  └─ Request targets decoy route? -> bounded tarpit delay + fake response
  └─ Normal traffic -> Continue
↓
6. Morpheus: RASP Hypervisor
  └─ Checks DML guards on wp_users & SSRF against 169.254.169.254
  └─ No violation -> Continue
↓
[ Phase 2: WordPress normal boot (plugins_loaded / init) ]
↓
Titan: Surface Policy Injection (CSP, HSTS, Permissions-Policy)
↓
Response delivered to the client

```

7.2 TRINITY XDR Incident Correlation & Response Flow

```

Security event at a sensor (e.g. Aegis WAF SQLi detection)
↓
VIS_Event_Bus::log() & EventFabric::ingest()
↓
XdrEvent creation with actor hash & correlation ID
↓
EventRepository::persist() -> stored in wp_vis_xdr_events
↓
IncidentEngine::correlate()
↓
Is there an active incident for the same actor/hash?
  └─ YES -> link in wp_vis_xdr_incident_events, update incident
  └─ NO -> create new incident in wp_vis_xdr_incidents + Attack Story
↓
PolicyEngine::evaluate() -> evaluate deterministic response rules
↓
ResponseEngine::execute()
↓
Countermeasure (e.g. Cerberus IP ban or session kill)
↓
Store in wp_vis_xdr_responses (including rollback JSON)
↓
EvidenceStore::append() -> Merkle digest chain in wp_vis_xdr_evidence

```

7.3 Integrity Scan & Baseline Commit Flow

```

User clicks "Start Scan" or Chronos cron fires
↓
AJAX 'vis_run_scan' to VIS_Dashboard_Ajax::handle_scan_bridge()
↓
VIS_Scanner_Engine_Omega::run_batch()
↓
Path iteration in the web root while excluding vis-vault-omega/
↓
For each file:
  └─ Calculate SHA-256 hash
  └─ VIS_Malware_Engine::inspect() (AST lexical, polyglot, SVG)
  └─ Compare with local baseline (NDJSON)
↓
Deviation or malware detected?
  └─ YES -> generate finding, optionally isolate in VIS_Quarantine_Store
  └─ NO -> file unchanged
↓
Update state JSON in vis-vault-omega/
↓
Return batch result as JSON to vis-scanner-client.js

```

```

↓
[Optional] User clicks "Approve Changes"
↓
AJAX 'vis_approve_changes'
↓
Recalculate Merkle root & commit to integrity_matrix.json

```

8. SHARED / CORE FILES

The following files form the **indispensable foundation** of the suite. They are classified as CORE / SHARED DEPENDENCY:

1. class-vis-bootstrapper.php

- **Class:** VIS_Bootstrapper
- **Role:** Master Kernel Bootstrapper.
- **Used by:** gedefense-wp.php.
- **Dependent modules:** all 20 modules and subsystems are initialized by the bootstrapper.

2. class-vis-schema.php

- **Class:** VIS_Schema
- **Role:** database & Vault-directory orchestration.
- **Used by:** gedefense-wp.php (activation hook & admin_init upgrade check).
- **Dependent modules:** Cerberus, XDR, Secure Downloads, Oracle, Rate Limiting.

3. class-vis-vault.php

- **Class:** VIS_Vault
- **Role:** low-level cryptography engine (Libsodium / OpenSSL AES-256-GCM).
- **Used by:** VIS_Key_Vault, VIS_Throne_Guard, VIS_Nemesis, VIS_Titan, VG_Vault_Bridge.

4. includes/core/class-vis-security.php

- **Class:** VIS_Security
- **Role:** central Zero-Trust utility library (client_ip, jailed_path, pinned_https_get, timing_safe_equals).
- **Used by:** Cerberus, Aegis, Zeus, Prometheus, Styx, Hades, Titan, Airlock, Dashboard Core.

5. includes/core/class-namespace-compatibility.php

- **Class:** VisionGaia\GeDefense\Core\NamespaceCompatibility
- **Role:** autoloader & interface bridge between the canonical namespace and legacy class names.
- **Used by:** VIS_Bootstrapper::register_autoloader().

6. includes/core/class-vis-event-bus.php

- **Class:** VIS_Event_Bus
- **Role:** standardized security-event emitter.
- **Used by:** Aegis, Cerberus, Prometheus, Nemesis, Ghost Trap, Airlock, Styx.

7. includes/core/class-vis-security-health.php

- **Class:** VIS_Security_Health
- **Role:** environment and PHP stability checks.
- **Used by:** Security Center, Overview Cockpit, Oracle, Kernel Sentinel.

8. includes/dashboard/class-vis-dashboard-assets.php

- **Class:** VIS_Dashboard_Assets
- **Role:** deterministic enqueue manager for styles and scripts.
- **Used by:** VIS_Dashboard_Core::inject_assets().

9. includes/xdr/class-xdr-event-fabric.php

- **Class:** VisionGaia\GeDefense\Xdr\EventFabric
- **Role:** central backbone for incident correlation and response triggers.
- **Used by:** EventBus and all sensor modules.

9. ARCHITECTURE RELATIONSHIPS (MERMAID DIAGRAMS)

9.1 Overall System Architecture

```
graph TD
    Client[HTTP Request] --> PluginEntry[gedefense-wp.php]
    PluginEntry --> Bootstrapper[VIS_Bootstrapper]

    subgraph "Phase 1: Pre-Flight Kernel (Zero-Overhead)"
        Bootstrapper --> Cerberus[Cerberus L0 Drop]
        Cerberus --> Zeus[Zeus 6G WAF]
        Zeus --> Aegis[Aegis DPI WAF]
        Aegis --> Prometheus[Prometheus Profiler]
        Prometheus --> Nemesis[Nemesis Deception]
        Nemesis --> Morpheus[Morpheus RASP]
        Morpheus --> Gorgon[Gorgon Telemetry]
    end

    end

    subgraph "Phase 2: Invariant Subsystems (plugins_loaded)"
        Titan[Titan Surface Policy]
        Hades[Hades Admin Cloaking]
        Airlock[Airlock Upload Guard]
        Styx[Styx Outbound Shield]
        ThroneGuard[ThroneGuard Superkey Gate]
        Chronos[Chronos Autopilot]
        Vault[Key Vault KMS]
    end

    end

    Morpheus --> WP[WordPress Application Runtime]
    WP --> Phase2[Phase 2 Invariants]
    Phase2 --> Titan
    Phase2 --> Hades
    Phase2 --> Airlock
    Phase2 --> Styx
    Phase2 --> ThroneGuard
    Phase2 --> Chronos
    Phase2 --> Vault

    subgraph "Data & Security Backbone"
        XDR[TRINITY XDR Engine]
        EventBus[VIS_Event_Bus]
        Schema[(MySQL DB: 10 tables)]
        VaultDir[(Sovereign Vault Storage)]
    end

    end

    Aegis --> EventBus
    Cerberus --> EventBus
    Prometheus --> EventBus
    EventBus --> XDR
    XDR --> Schema
    Chronos --> VaultDir
    Vault --> VaultDir
```

9.2 Dashboard Architecture

```
graph TD
    User[Administrator] --> Core[VIS_Dashboard_Core]
    Core --> MenuMatrix[register_menu_matrix]
    MenuMatrix --> Suite[admin.php?page=vgt-suite]
    MenuMatrix --> ThronePage[admin.php?page=vgt-throneguard]
    MenuMatrix --> LoginPage[admin.php?page=vgt-loginpager]

    Suite --> ViewRouter[VIS_Dashboard_View::render]
    ViewRouter --> Sidebar[view-sidebar.php]
    ViewRouter --> ActiveTab[Tab Switcher]

    ActiveTab --> TabOverview[view-overview.php]
    ActiveTab --> TabZeus[view-zeus.php]
    ActiveTab --> TabAegis[view-aegis.php]
    ActiveTab --> TabTitan[view-titan.php]
    ActiveTab --> TabTrinity[view-trinity.php]
    ActiveTab --> TabSecurity[view-security_center.php]
    ActiveTab --> TabOthers[21 additional views...]

    Core --> AssetManager[VIS_Dashboard_Assets::enqueue]
    AssetManager --> GlobalCSS[vis-dashboard.css / modern.css]
    AssetManager --> TabCSS[Tab-specific style.css]
    AssetManager --> TabJS[Tab-specific script.js]

    TabJS --> AjaxHub[VIS_Dashboard_Ajax]
    AjaxHub --> BackendHandlers[Backend Services & Scanners]
```

9.3 Module Dependencies

```
graph LR
    subgraph "Core Utilities"
```

```

Security[VIS_Security]
VaultCore[VIS_Vault]
Health[VIS_Security_Health]
EventBus[VIS_Event_Bus]
end

subgraph "Enforcement & Defense"
Cerberus[Cerberus]
Aegis[Aegis]
Zeus[Zeus]
Prometheus[Prometheus]
Nemesis[Nemesis]
GhostTrap[Ghost Trap]
Airlock[Airlock]
Styx[Styx]
Titan[Titan]
ThroneGuard[ThroneGuard]
end

Security --> Cerberus
Security --> Aegis
Security --> Zeus
Security --> Styx
Security --> Titan
VaultCore --> ThroneGuard
VaultCore --> Nemesis
VaultCore --> Titan

Aegis --> Cerberus
Prometheus --> Cerberus
GhostTrap --> Cerberus

Aegis --> EventBus
Zeus --> EventBus
Prometheus --> EventBus
EventBus --> XDR[TRINITY XDR]
XDR --> Cerberus

```

9.4 Backend/API/Database Data Flow

```

sequenceDiagram
    autonumber
    participant Browser as Client / Attacker
    participant WAF as Aegis / Zeus
    participant EventFabric as TRINITY XDR EventFabric
    participant Repo as EventRepository
    participant Engine as IncidentEngine
    participant DB as MariaDB / MySQL
    participant Cerberus as Cerberus L0

    Browser->>WAF: Malicious request (SQLi / XSS)
    WAF->>WAF: Recursive normalization & DFA match
    WAF->>EventFabric: ingest(XdrEvent)
    EventFabric->>Repo: persist()
    Repo->>DB: INSERT INTO wp_vis_xdr_events
    EventFabric->>Engine: correlate(event)
    Engine->>DB: SELECT/UPDATE wp_vis_xdr_incidents
    Engine->>Cerberus: trigger_ban(ip, reason)
    Cerberus->>DB: INSERT INTO wp_vis_apex_bans
    Cerberus->>Cerberus: Cache in APCu / RAM
    WAF->>Browser: HTTP 403 Forbidden (Exploit Terminated)

```

10. FILE REFERENCES

For unambiguous technical traceability, the most important class definitions are mapped to their source paths below:

Class / Interface	File	Relevant Methods
VIS_Bootstrapper	class-vis-bootstrapper.php	register_autoloader(), engage_phase_1(), engage_phase_2()
VIS_Schema	class-vis-schema.php	enforce(), XDR_SCHEMA_VERSION, DOWNLOAD_SCHEMA_VERSION
VIS_Vault	class-vis-vault.php	encrypt(), decrypt(), get_primary_key()
VIS_Security	includes/core/class-vis-security.php	client_ip(), pinned_https_get(), jailed_path()
NamespaceCompatibility	includes/core/class-namespace-compatibility.php	register(), autoload(), canonical()

Class / Interface	File	Relevant Methods
VIS_Module_Integrity	includes/core/class-vis-module-integrity.php	verify_all(), calculate_merkle_root()
VIS_Module_Registry	includes/core/class-vis-module-registry.php	all(), enabled(), path()
VIS_Event_Bus	includes/core/class-vis-event-bus.php	init(), log(), emit()
VIS_Security_Health	includes/core/class-vis-security-health.php	evaluate(), score()
VIS_Security_Center	includes/core/class-vis-security-center.php	run_diagnostics(), get_snapshot()
VIS_Cerberus	includes/modules/cerberus/class-vis-cerberus.php	instance(), ban_ip(), unban_ip(), check_request()
VIS_Aegis	includes/modules/aegis/class-vis-aegis.php	__construct(), inspect(), normalize()
VIS_Zeus	includes/modules/zeus/class-vis-zeus.php	get_instance(), filter_request()
VIS_Prometheus	includes/modules/prometheus/class-vis-prometheus.php	evaluate_request(), record_threat()
VIS_Nemesis	includes/modules/nemesis/class-vis-nemesis.php	deploy_deception(), trap()
Vis_Morpheus	includes/modules/morpheus/class-vis-morpheus.php	engage(), get_hypervisor()
VIS_Titan	includes/modules/titan/class-vis-titan.php	__construct(), enforce_protocols(), handle_policy_action()
VIS_Throne_Guard	includes/modules/throneguard/class-vis-throne-guard.php	get_instance(), handle_claim_master(), handle_unlock()
DownloadManager	includes/modules/downloads/class-secure-download-manager.php	boot(), serve(), registerAction()
VIS_Dashboard_Core	includes/dashboard/class-vis-dashboard-core.php	register_menu_matrix(), inject_assets()
VIS_Dashboard_View	includes/dashboard/class-vis-dashboard-view.php	get_tabs(), render()
VIS_Dashboard_Settings	includes/dashboard/class-vis-dashboard-settings.php	process_mutations(), handle_standard_config()
VIS_Dashboard_Assets	includes/dashboard/class-vis-dashboard-assets.php	enqueue()
VIS_Dashboard_Ajax	includes/dashboard/class-vis-dashboard-ajax.php	mount_endpoints(), handle_scan_bridge()
EventFabric	includes/xdr/class-xdr-event-fabric.php	boot(), ingest(), record_event()
VIS_Scanner_Engine_Omega	includes/scanner/class-vis-scanner-engine.php	run_batch(), commit_baseline()

11. ARCHITECTURE OBSERVATIONS

The following architectural characteristics were identified during code analysis:

1. Inactive / historical views in the dashboard directory:

- includes/dashboard/views/view-aos.php: a bridge view for All-In-One Security (AIOS) that is not registered in the active \$tabs tab router. It still contains an inline <style> tag.
- includes/dashboard/views/view-myrmidon.php: an unused Device Trust view with inline event handlers (onclick, onsubmit) that is not routed through the menu.
- *Recommendation:* In the medium term, these files should be moved to a legacy/ archive or removed entirely to reduce code baggage.

2. Binary archive artifacts in the source tree:

- includes/modules/gorgon/gorgon.tar.gz (7,251 bytes)

- `includes/VisionGaiaSE0/VisionGaiaSE0.zip` (43,980 bytes)
- *Recommendation:* Keeping ZIP and TAR archives inside the PHP source tree should be replaced by an external release or package-management process.

3. Test-structure discrepancy:

- The `tests/Unit/` directory is empty.
- The project's complete test coverage (28 highly specialized regression tests for WAF, XDR, Titan, Scanner, Morpheus) resides in the `scripts/` directory.
- *Recommendation:* The test suite in `scripts/` works very well and completely independently of PHPUnit; `tests/` can be cleaned up or formally populated with wrapper classes.

4. Strong cross-module dependencies:

- `VIS_Aegis` and `VIS_Prometheus` couple directly to `VIS_Cerberus::ban_ip()`.
- `VIS_Zeus` has a direct bridge to XDR (`Zeus_Xdr_Bridge`).
- `VIS_Titan` reads quarantine certificates from `Airlock`.
- *Security rationale:* These couplings are intentional because, in L0-L1 defensive scenarios, WAF decisions must be forwarded atomically and directly to the blocking filter to minimize latency.

Document generated and verified for GeDefense WP - Open Core (Standard WordPress Plugin v8.1.0).

Part V - Sources and version note

This manual was created from the supplied **GeDefense WP - Open Core 8.1.0** source state and its `ARCHITECTURE.md`. Settings and feature descriptions refer to this version. After later code changes, both the architecture map and the manual must be reviewed and updated.